Cardiff Metropolitan University

ICBT Campus

CIS6003 Advanced Programming

Individual Written Report (WRIT1)

Sunrise Dental Clinic

A three-tier distributed appointment and patient management system

Thanujaya Hasaranga Perera

Registration number: st20374257

Repository: github.com/Thanujaya448/sunrise-dental-clinic

Word count (body, excluding references and appendices): 4,967

Submission date: _____

Contents

1. Introduction

Sunrise Dental Clinic manages appointments, patient records and billing on paper. The brief lists six functions the clinic believes it needs, but it also lists four problems it is actually suffering: double bookings, lost patient records, long waiting times, and billing errors. Those four failures, not the six functions, were treated as the specification.

The distinction matters, because each failure implies a requirement the six functions never mention. Double bookings can only occur if an appointment occupies a period rather than an instant, so every treatment must carry a duration and the system needs a defined, enforced rule for what counts as a clash. Lost records imply that a patient is a persistent entity referenced by each appointment, not fields copied into every booking. Long waiting times suggest that refusing an unavailable slot is half a solution — the system should offer an alternative. Billing errors point to prices held in maintained data rather than in code.

This report describes a three-tier distributed system built to address them: a browser client and a Java Swing client over one REST API, a Spring Boot business tier, and a MySQL 8 data tier in which several rules are enforced by triggers and a stored procedure. It sets out the assumptions, the design decisions with the alternatives rejected, the testing strategy and its evidence, and an honest evaluation of what the system does not do.

2. Assumptions and derived requirements

The brief invites assumptions on system design and access permissions. Thirteen were made, each traced to a line in the brief, one of the four stated failures, or a marking criterion. The significant ones follow.

Roles (ASM-01, ASM-02). The brief says only "authorised staff". Three roles were assumed — Receptionist, Dentist and Administrator — because a shared account destroys accountability, and an audit trail that cannot name a person is useless when a booking is disputed. The role determines the menu, but authorisation is enforced in the business tier: hiding a button is not a control.

Patient persistence (ASM-03). The brief requires *new* patients to be registered, which is only meaningful if returning ones are looked up. Patient is therefore a first-class entity

with a generated number, referenced by each appointment. Copying details into every appointment would recreate the lost-records failure.

Treatment durations (ASM-07). The brief's largest omission. Without a duration per treatment, "double booking" can only mean the identical minute, which is not the problem described. Each treatment type carries a standard duration, and an appointment's end time is derived from it rather than typed.

The clash rule (ASM-08). Two appointments clash when their ranges, each widened by a ten-minute turnaround buffer, intersect for the same dentist on the same date. Cancelled and no-show appointments release the slot.

Discounts (ASM-12). Four categories — none, senior 10%, returning patient 5%, staff and family 15% — with the most generous applying and no stacking. This was deliberate: it gives the Strategy pattern real work rather than a token single implementation.

Scope (ASM-13). No tax is modelled. Stated here as a scoping decision rather than left as an unexplained gap.

3. Requirements modelling and UML

Six diagrams were produced: a use case diagram, a domain class diagram, a design class diagram with a pattern-focused companion, and three sequence diagrams. All are written in PlantUML, with the `.puml` sources committed beside the rendered images, so diagrams are versioned, diffable and reviewable in a pull request. A binary drawing exported from a graphics tool cannot be diffed, and in practice stops being updated.

3.1 Use case diagram

Twenty-one use cases across four actors. `Receptionist`, `Dentist` and `Administrator` inherit from an abstract `Staff User`, so behaviour every signed-in user shares is modelled once. A `Notification Gateway` appears as a secondary actor: it is a system the clinic depends on, not a person using it.

Notation follows Fowler (2003). `<<include>>` marks behaviour the base case **always** performs — booking always records an audit entry — while `<<extend>>` marks optional behaviour guarded by a condition: *Suggest Alternative Slot* extends *Book Appointment* only when the requested slot is unavailable. The arrows point in opposite directions, a common error, so the diagram carries a legend stating both.

One decision is worth defending. Authentication is a *precondition* of every use case, not an `<<include>>` on each: twenty-one include arrows to *Log In* would be defensible and useless, adding no information while obscuring the relationships that carry some. It is recorded in each specification instead.

3.2 Domain class diagram

The domain model is an **analysis** model: it describes the problem domain, not the Java classes implementing it (Larman, 2004). `DentistSchedule` appears because a dentist's working day is a concept the clinic reasons about, though it has no class in the code, where the behaviour is a private helper in `AppointmentService`. That is not an inconsistency — a domain model mirroring the implementation would add nothing the design class diagram does not show.

Four relationship kinds are used deliberately, each tied to its consequence in the schema. **Multiplicity** is stated at both ends of every association. **Navigability** is shown by the arrowheads and is not decoration: an `Appointment` holds a reference to its

`Patient`, so "whose appointment is this?" is answered directly while the reverse is a query, and drawing every association bidirectional would claim references the design does not hold. **Composition** joins `Bill` and `BillLine` — a line has no meaning apart from its bill, and `bill_line`'s foreign key is `ON DELETE CASCADE`, so deleting a bill removes its lines by definition rather than by convention. **Aggregation** joins `DentistSchedule` and `Appointment` — a schedule groups appointments that exist independently of it, and `appointment`'s key to `patient` is `ON DELETE RESTRICT`, because deleting a patient must never silently destroy their history. Recording the UML relationship and its SQL consequence together is what stops the model being decorative.

3.3 Design class diagram and sequence diagrams

The design class diagram shows the three physical tiers and the pattern participants in each. It was **redrawn from the implemented system** partway through the project: the first version was design intent, and the code then diverged from it, naming four controllers, three repositories, a mapper and a report factory that were never built. Submitting it would have meant submitting a diagram of a system that does not exist. The revised version and the three sequence diagrams revised with it name only classes present in the repository, and §5.5 explains each divergence as a decision rather than an omission.

The sequence diagrams cover logging in, booking against a clash, and generating a bill. Each marks the HTTP boundary explicitly and shows the failure paths beside the success path — the value of the booking diagram is precisely that it shows what happens when the trigger rejects the insert.

Together the three views answer different questions and check each other: the use case diagram fixes *what* the system must do, the class diagrams *what it is made of*, and the sequence diagrams *what happens in what order*. It was the third that exposed the drift in the second.

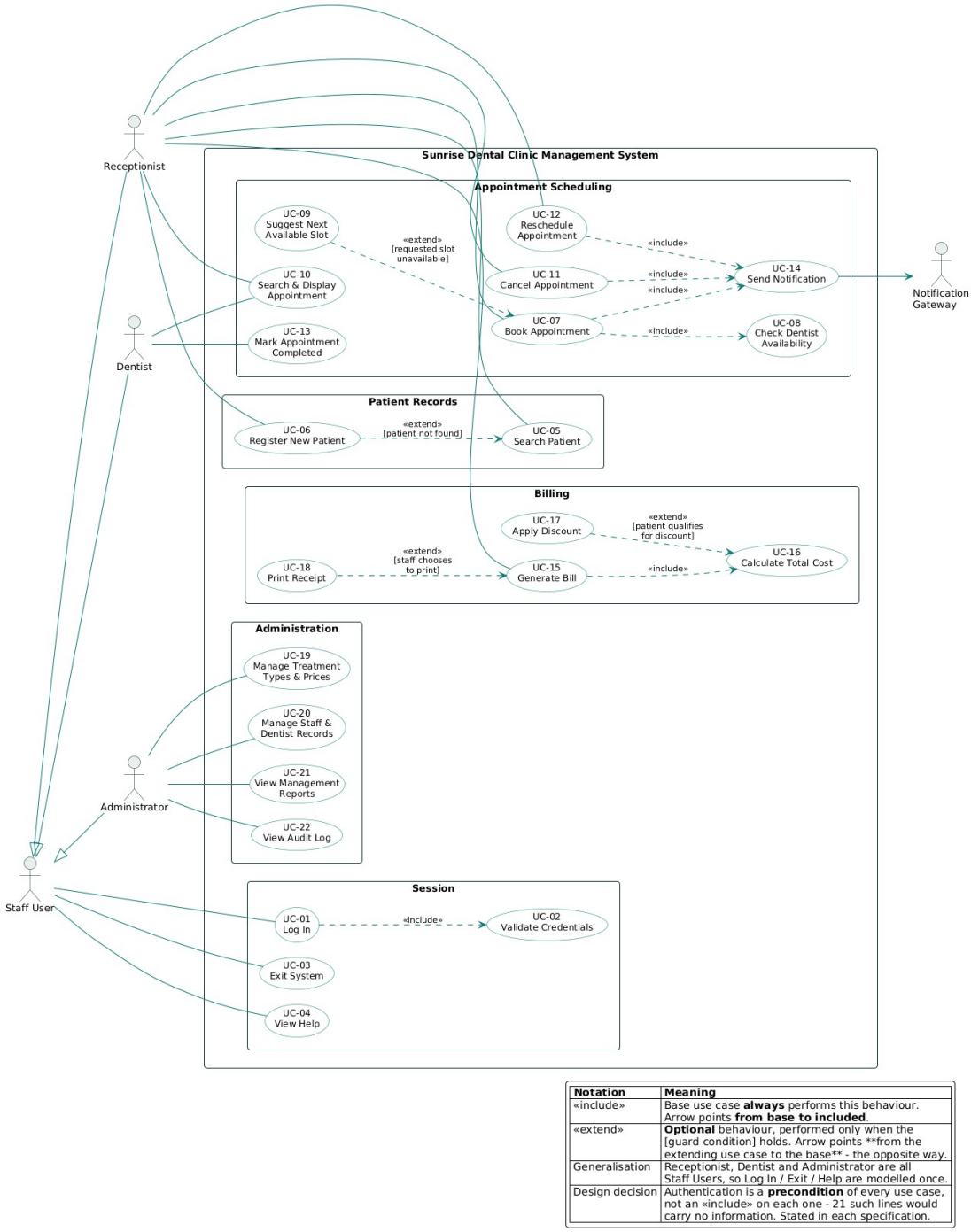
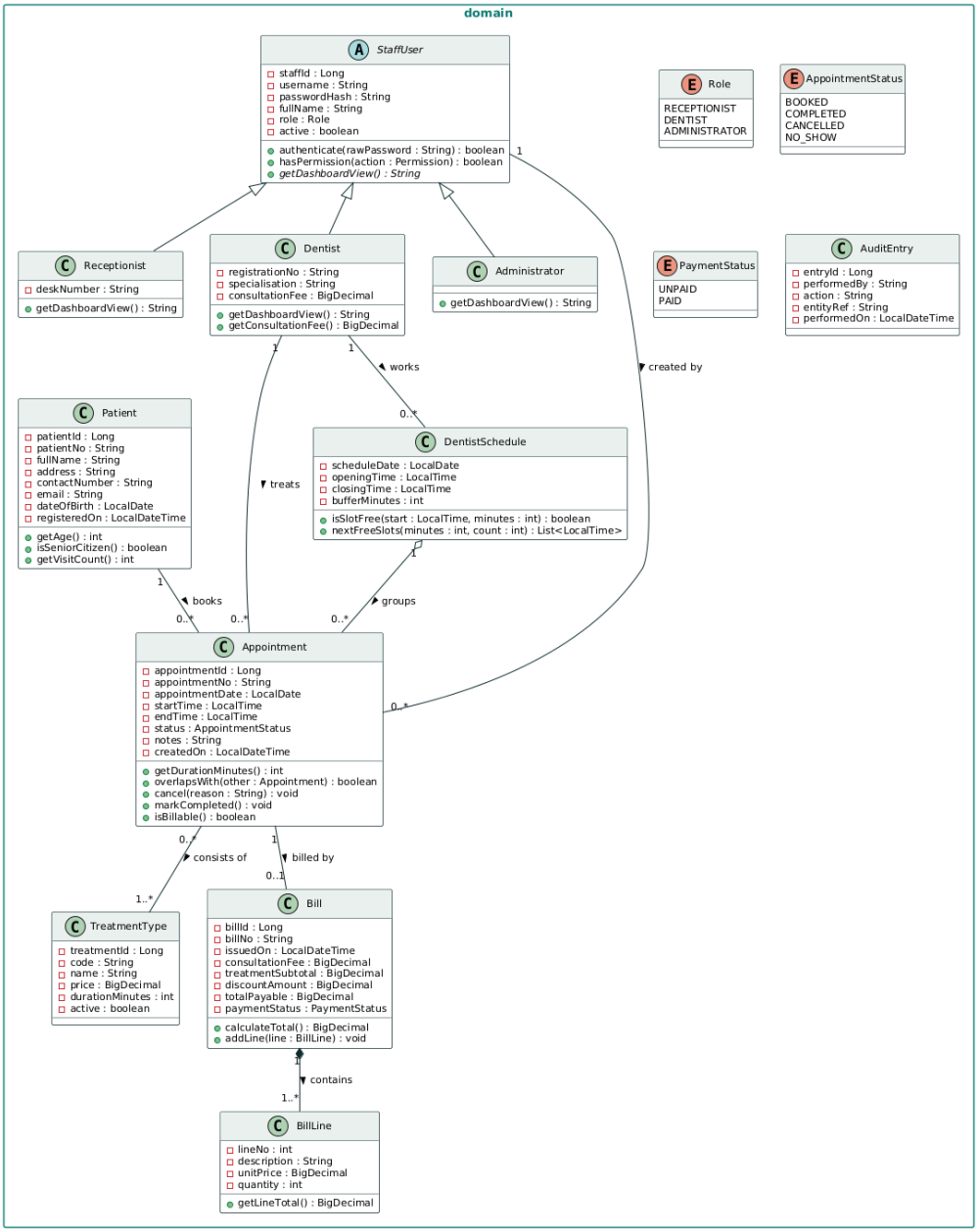


Figure 1: Use case diagram (Task A).



Symbol	Relationship	Why it is used here
—▷	Generalisation	Receptionist / Dentist / Administrator **are** StaffUsers
———	Association	Independent objects that know about each other, labelled with multiplicity at both ends
—◆	**Composition**	Bill ◆ BillLine - a line has no meaning without its bill and is destroyed with it (owned lifetime)
—◇	**Aggregation**	DentistSchedule ◇ Appointment - the schedule groups appointments that exist independently of it

Figure 2: Domain class diagram, showing navigability, multiplicity, composition and aggregation.

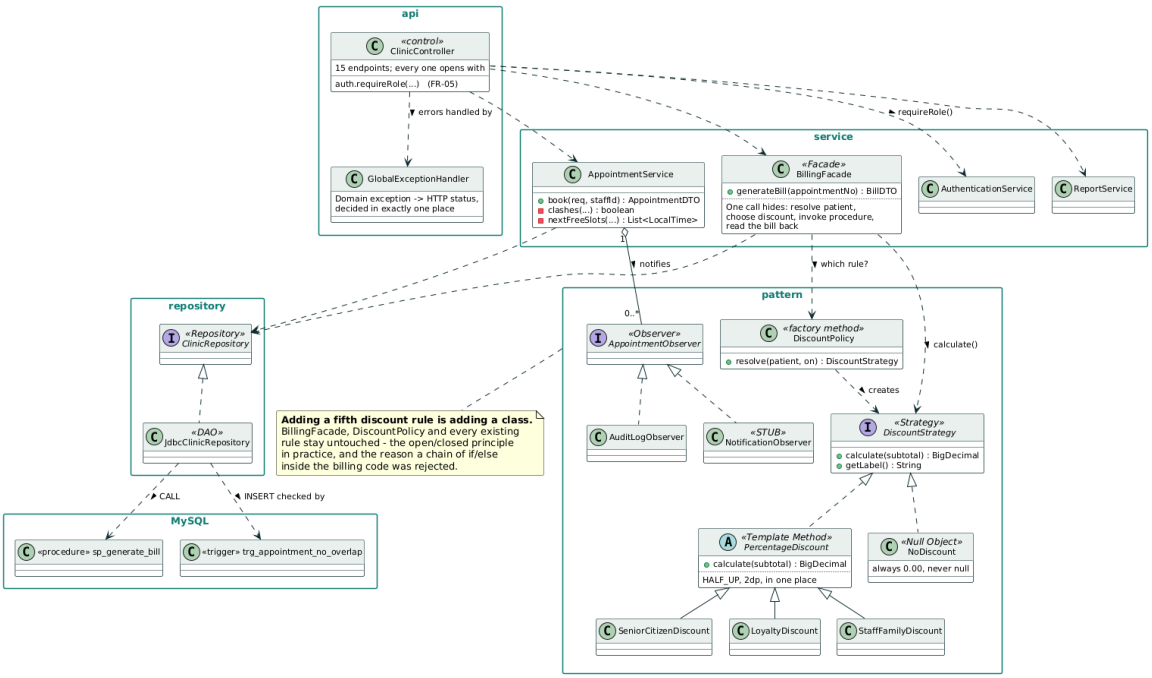


Figure 3: Design class diagram, business tier. Full three-tier version at Appendix B.

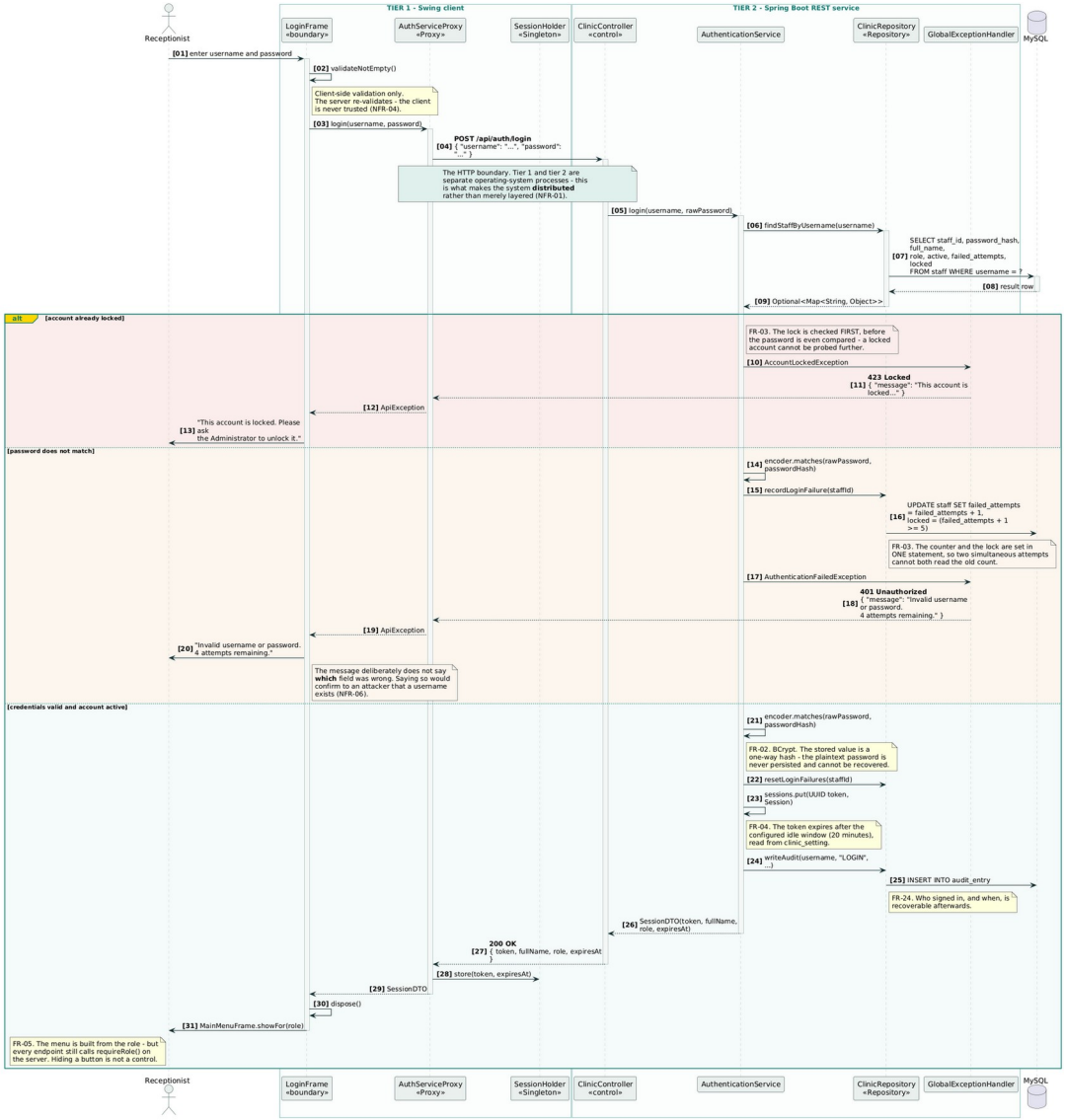


Figure 4: Sequence: UC-01 Log In.

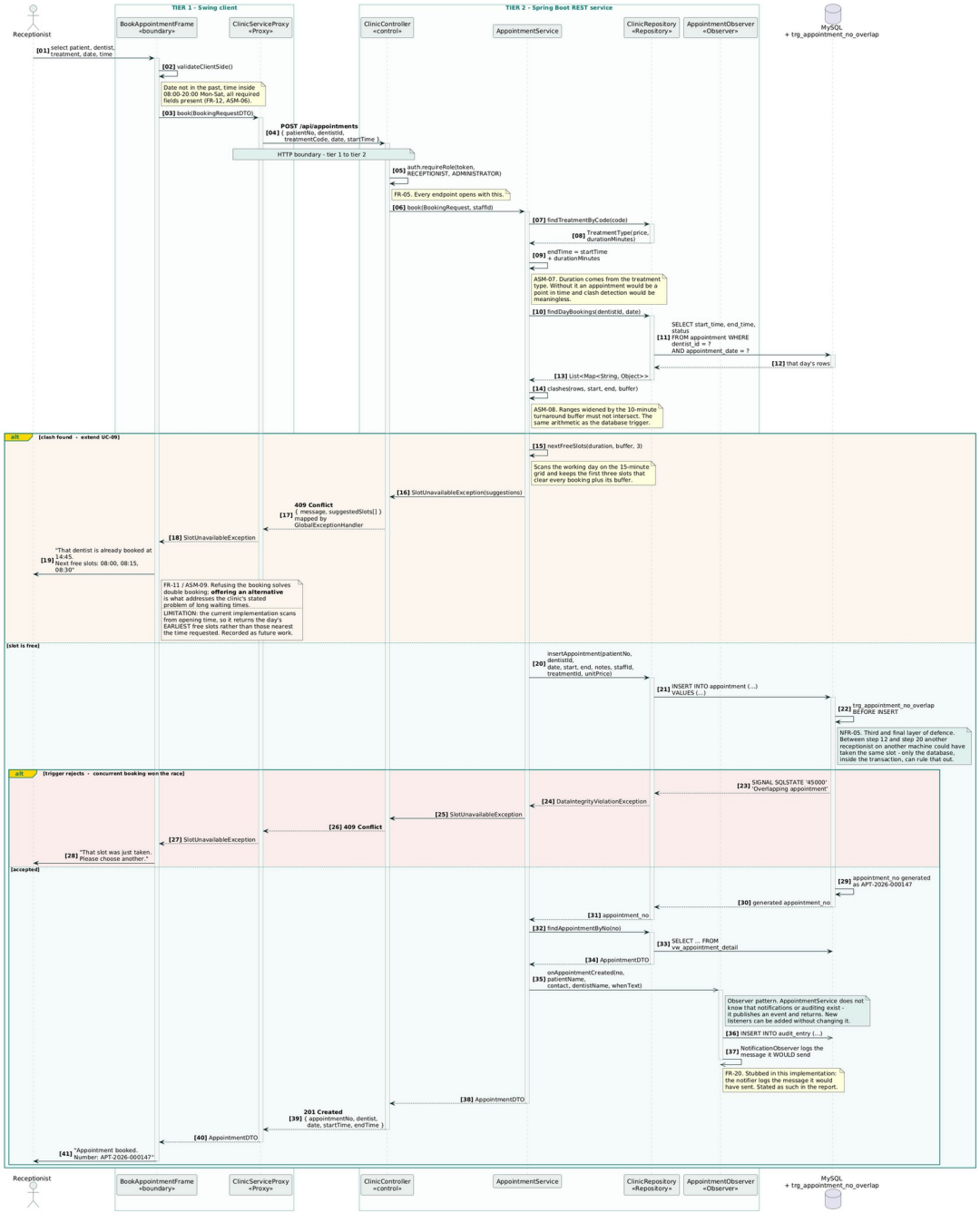


Figure 5: Sequence: UC-07 Book Appointment.

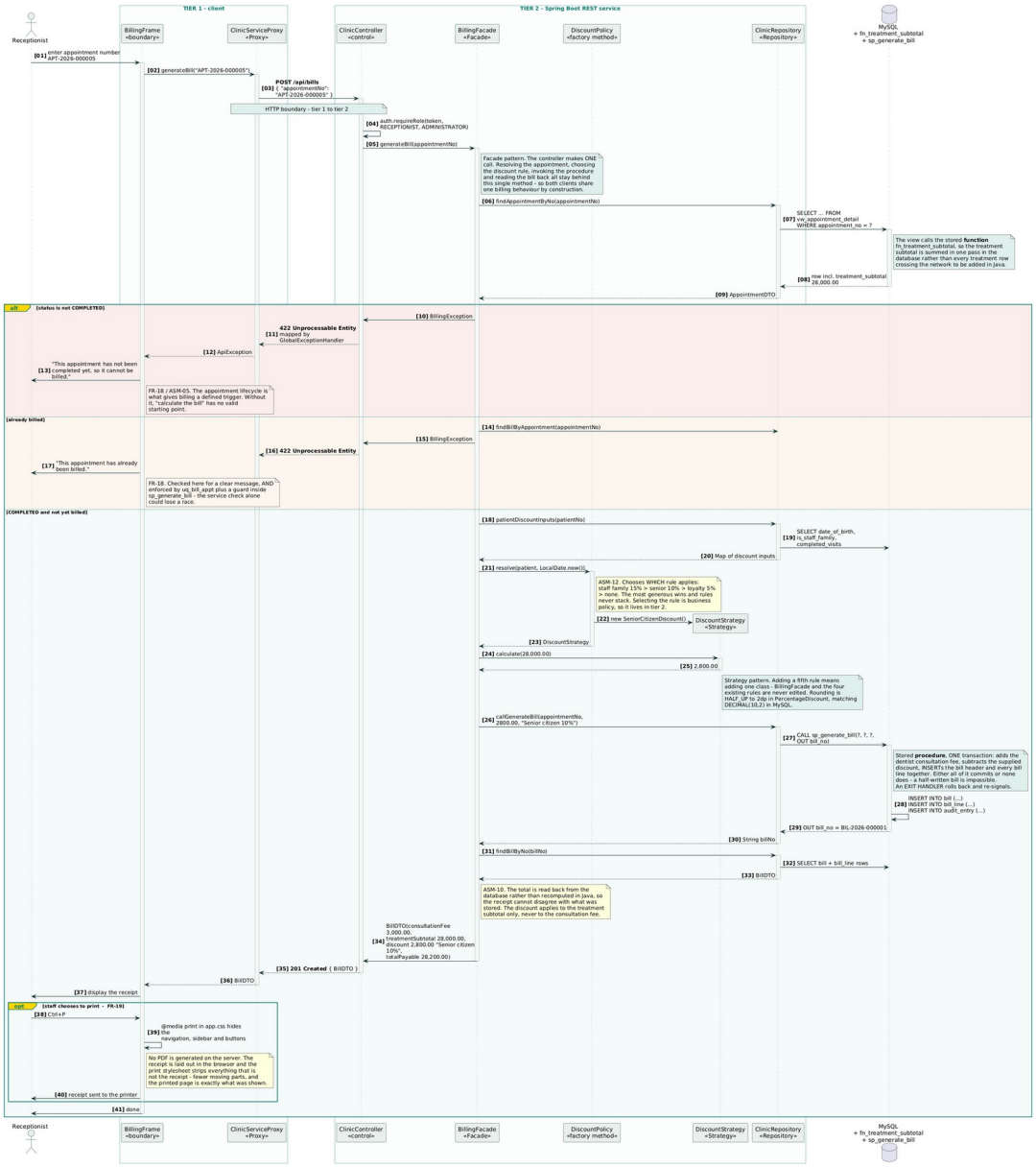


Figure 6: Sequence: UC-14 Generate Bill.

4. Three-tier distributed architecture

The system runs as three operating-system processes: a client, a Spring Boot REST service on port 8080 (VMware, 2024), and MySQL 8 on port 3306 (Oracle, 2024), whose twelve tables are in third normal form (Codd, 1970; Date, 2003). The distinction that matters is between *layers* and *tiers*. Packaging code into presentation, business and data layers inside one executable is a logical separation a careless import can undo. An HTTP boundary makes it physical: the client cannot reach a repository or a SQL statement even if a developer wanted it to, because those classes are not in its process (Fowler, 2002).

4.1 Separation proved rather than asserted

Any project can draw three boxes and claim its tiers are separated. Two things here make the claim testable.

Two independent clients share one API. A browser client served from the service's own `static/` directory and a Java Swing desktop client call the same fifteen endpoints, and neither holds a business rule. Had one leaked into the browser — the clash arithmetic, the discount percentages, the billing total — the Swing client could not exist without duplicating it, and the two would diverge at the first change. The second client was built partly as a check.

The business tier runs with no database. The 69 service-layer tests in §6 execute `AuthenticationService`, `AppointmentService` and `BillingFacade` against a hand-written in-memory repository, with no MySQL, no JDBC driver and no Spring context. Had a rule been written into a SQL string rather than into Java, those tests could not run. A green suite is evidence of the boundary, not only of correctness.

4.2 Where each rule lives, and one deliberate duplication

Validation is duplicated intentionally. The client checks required fields and opening hours so the receptionist gets an immediate response; the service re-checks everything because the client is never trusted (NFR-04). The service is authoritative, and Figure 21 shows a dentist's own session receiving HTTP 403 from a report endpoint although the tab was never displayed.

The clash rule is duplicated more contentiously. `AppointmentService` checks for a clash *and* `trg_appointment_no_overlap` enforces it inside the transaction. Two objections were weighed. Against: the rule exists in two languages and could diverge, and business logic in the data tier is criticised as harder to test and version. For: between the service's check and its `INSERT`, another receptionist on another machine can commit the same slot, and no application code can close that window — only a constraint inside the transaction can (NFR-05). The compromise makes the divergence risk visible rather than theoretical: the same arithmetic is asserted in Java by `AppointmentOverlapTest` and against a real MySQL server by the CI pipeline on every push, so if the two disagree the build fails.

4.3 What the architecture costs

Every operation pays a network round trip, the system cannot run offline, and deployment means three components rather than one. Sessions held in a `ConcurrentHashMap` are lost on restart and would not survive a second instance — a limitation §8 returns to. For a single clinic these costs are acceptable, and Fowler's (2002) advice against distributing objects needlessly is why the boundary is one coarse hop per user action rather than a chatty interface.

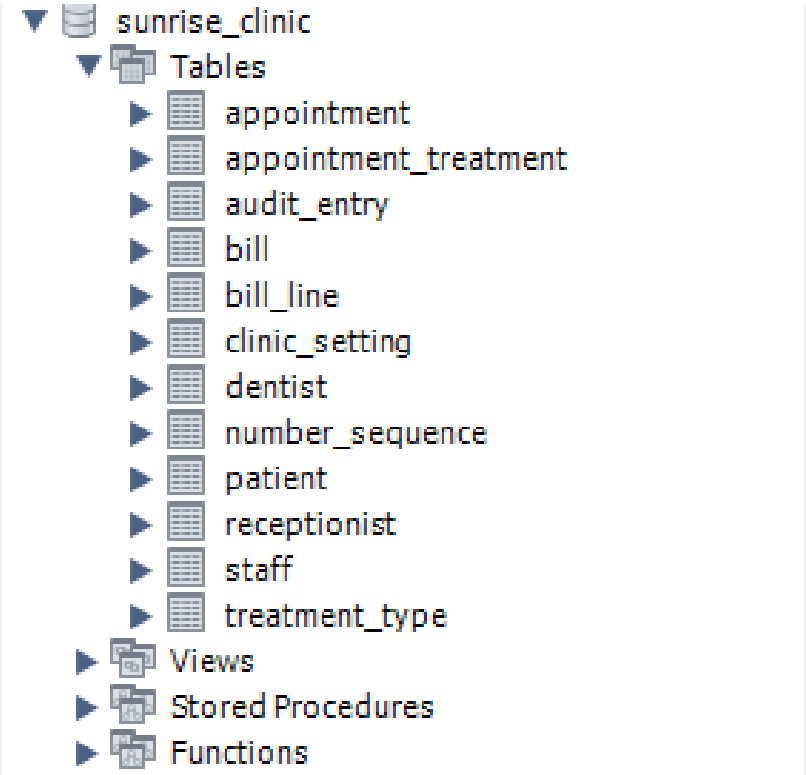


Figure 7: Tier 3: twelve tables in third normal form.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

check_name	found	result
Tables (expect 12)	12.00	PASS
Views (expect 5)	5.00	PASS
Triggers (expect 4)	4.00	PASS
Function + procedure (expect 2)	2.00	PASS
Foreign keys (expect 9)	9.00	PASS
Staff seeded (expect 5)	5.00	PASS
Dentists seeded (expect 3)	3.00	PASS
Treatments seeded (expect 10)	10.00	PASS
Patients seeded (expect 5)	5.00	PASS
Appointments seeded (expect 10+)	11.00	PASS
Appointment numbers generated	0.00	PASS
Patient numbers generated	0.00	PASS
fn_treatment_subtotal returns 28000	28000.00	PASS
BCrypt hashes stored (expect 5)	5.00	PASS

Figure 8: Health check: all fourteen rows PASS.

Figure 9: The overlap trigger refusing a clash. The error is the expected result.

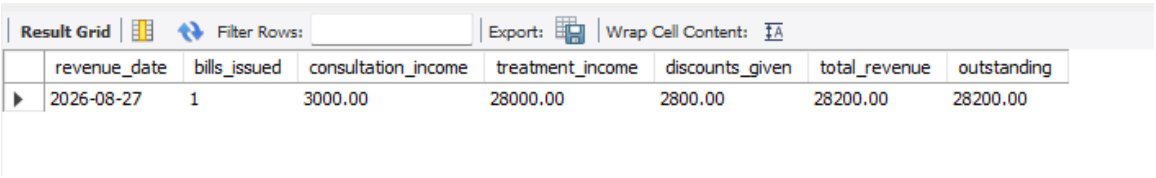


Figure 10: sp_generate_bill: 3,000 + 28,000 - 2,800 = 28,200.00.

5. Design patterns applied and evaluated

Patterns were selected because a force in this system called for one, not to satisfy a checklist. Gamma *et al.* (1994) warn that applying a pattern where the problem does not exist adds indirection without benefit, and §5.5 records two patterns considered and rejected on exactly that ground.

5.1 Strategy and Null Object — the discount rules (behavioural)

Four discount categories apply: none, senior 10%, returning patient 5%, staff and family 15% (ASM-12). Each is encapsulated behind a `DiscountStrategy` interface with one `calculate` method, and `DiscountPolicy.resolve()` selects the one that applies.

The alternative rejected was a chain of `if`/`else if` inside `BillingFacade`. It is shorter and needs no interface. It was rejected because the discount rules vary independently of how a bill is assembled: under the if/else design every new rule edits a method that is already tested and working, putting the passing rules at risk each time. With Strategy, adding a fifth rule is adding a class; the facade, the policy and the four existing rules are untouched. That is the open/closed principle in a form that can be demonstrated rather than asserted, and the eighteen tests in `DiscountStrategyTest` prove each rule in isolation, which an if/else chain would not permit.

Null Object completes it. "No discount applies" is itself a strategy returning `0.00` rather than `resolve()` returning `null`. Fowler (2002) calls this Special Case: the absent value is an object that behaves correctly, so no caller tests for it. `BillingFacade` contains no null check, and no future caller can forget one.

What it cost. Six classes where a dozen lines would have worked, and one more indirection before a reader sees the arithmetic. For four rules that is a fair exchange; for a single fixed rule it would be over-engineering and the pattern would not have been used.

An abstract `PercentageDiscount` sits between the interface and the three percentage rules. This is **Template Method**: the invariant part — multiply, round `HALF_UP` to two places, match `DECIMAL(10,2)` in MySQL — is fixed in the superclass and only the rate and label vary. Without it the three subclasses would round independently and could drift apart, producing bills that fail to reconcile by a cent.

5.2 Observer — appointment events (behavioural)

When an appointment is booked, two things must happen besides the booking: the patient is emailed a confirmation, and the event reaches the audit trail (FR-20, FR-24). `AppointmentService` publishes to a list of `AppointmentObserver` implementations injected by Spring and knows about neither.

The alternative rejected was calling the mailer and the audit writer directly from `book()`, which couples the clinic's most important method to two concerns that are not booking and makes every new listener edit it again. Under Observer a third listener is a new `@Component` and `book()` does not change. The tests show the decoupling: `AppointmentService` is built with a recording observer in one test and an empty list in another, and booking succeeds in both.

`NotificationObserver` sends a real confirmation over SMTP (Figures 23 and 24), and three decisions in it are worth defending. It is `@Async`, because a slow or briefly unreachable mail server must not make the receptionist wait for a booking that has already committed — the log line in Figure 23 is written on a `task-1` thread, not the request thread. Every failure is caught and logged rather than rethrown: an undelivered confirmation is a nuisance, but an exception unwinding a committed appointment would be a defect, so notification is kept out of the booking transaction by construction. And sending is switched by configuration rather than by code, so the tests and the CI pipeline exercise the same class that ships, with delivery off.

What it cost. The flow is harder to trace in a debugger, because nothing in `book()` names who will be called. That is the standard price of indirection.

5.3 Facade — billing (structural)

Generating a bill means resolving the appointment, checking it is completed and not already billed, building a `Patient` from its discount inputs, selecting a strategy, invoking a stored procedure and reading the bill back. `BillingFacade.generateBill(appointmentNo)` is the only method the controller calls. **The alternative rejected** was letting the controller orchestrate those steps: the sequence would then be duplicated in both clients, and the moment a rule changed one of them would be missed. Because the facade holds it, both clients share one billing behaviour by construction.

5.4 Proxy, Singleton, Repository and Adapter

Proxy *(structural)* appears in both clients. `ClinicServiceProxy` and the browser's `request()` expose the service as though it were local, hiding the HTTP call, the `Authorization` header and the mapping of status codes to meaning.

Singleton *(creational)* holds the session in each client. It is justifiably criticised as global state that complicates testing (Martin, 2017), and that criticism applies here. It was accepted only because the scope is one client process holding one signed-in user, and threading the token through every constructor would obscure the design for no practical gain. It is not used anywhere in the business tier.

Repository and DAO separate the service layer from SQL; the measurable benefit is in §6. One interface was used rather than five, which keeps the wiring visible at twelve tables; Fowler (2002) would split it per aggregate as the system grew. **Adapter / DTO** records form the API contract and deliberately do not mirror the tables, so no client learns a primary key or a column name and the schema can change without breaking tier 1.

5.5 Patterns considered and rejected

Factory Method with Template Method for reports was in the original design: a `ReportFactory` producing `AbstractReport` subclasses. The implemented system maps a fixed `ReportType` enum to a view name instead. Four read-only queries do not justify four classes of hierarchy, and the enum makes SQL injection structurally impossible — a caller cannot supply a view name that is not already a constant. The pattern would have added machinery and removed a security property.

An ORM (JPA/Hibernate) was rejected for the data tier. Several rules live in a trigger and a stored procedure, and an ORM's caching and generated SQL work against database-side logic. Spring JDBC keeps the interaction explicit, at the cost of hand-written row mapping.

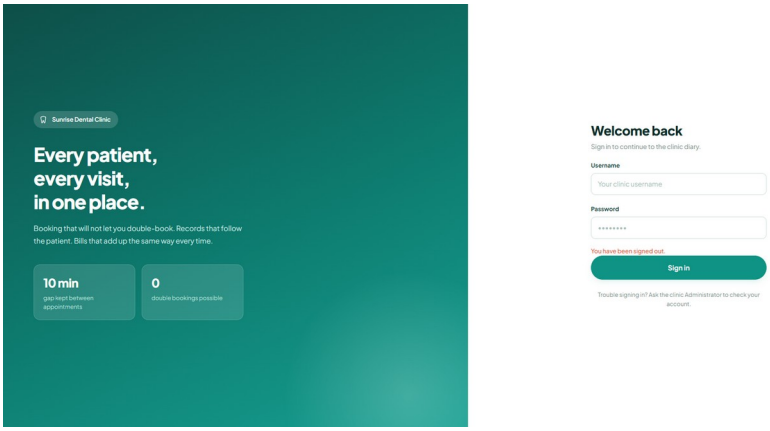


Figure 11: Sign-in screen.

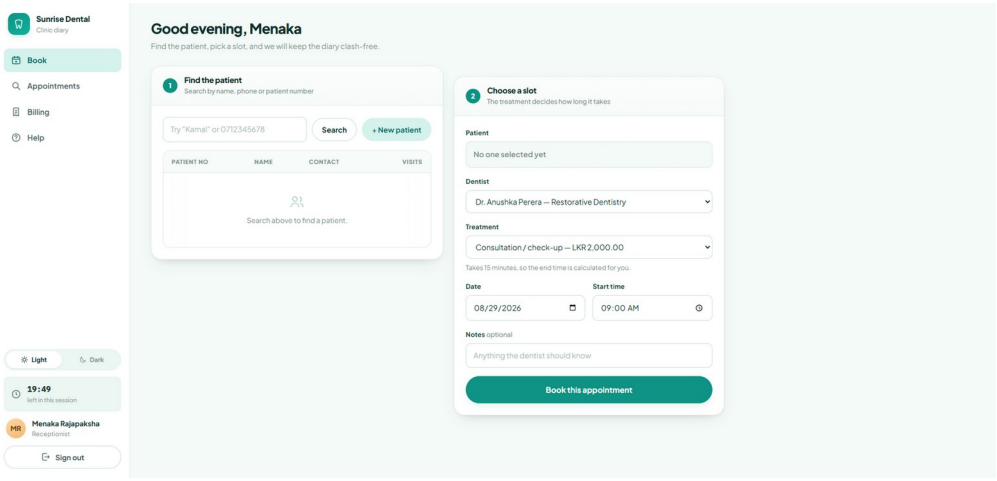


Figure 12: Receptionist dashboard.

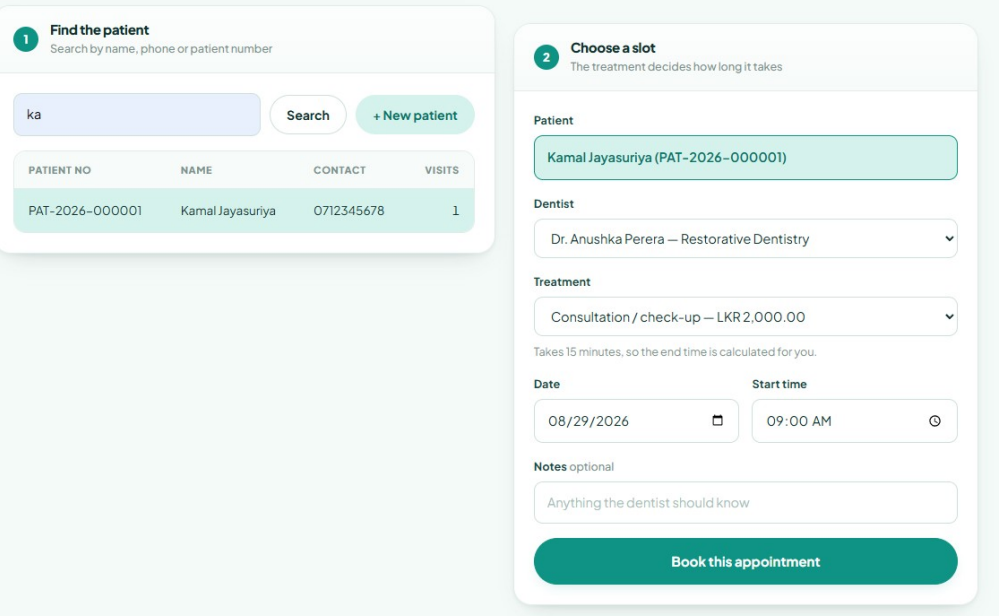


Figure 13: Booking form; the end time is derived from the treatment duration.

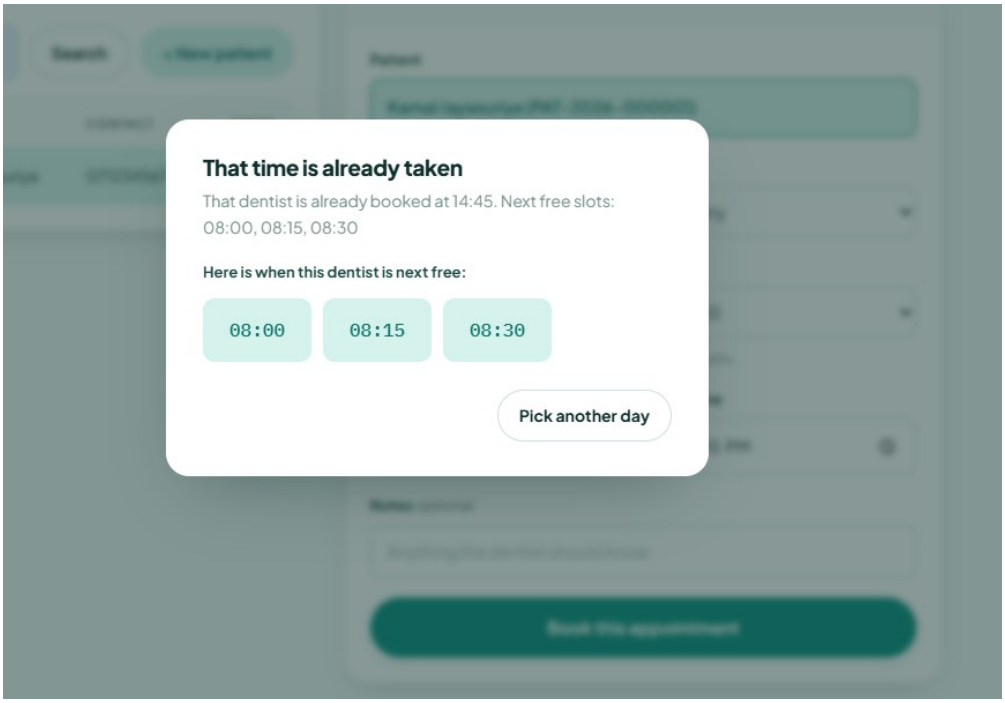


Figure 14: FR-10 and FR-11: the clash is refused and three alternatives offered.

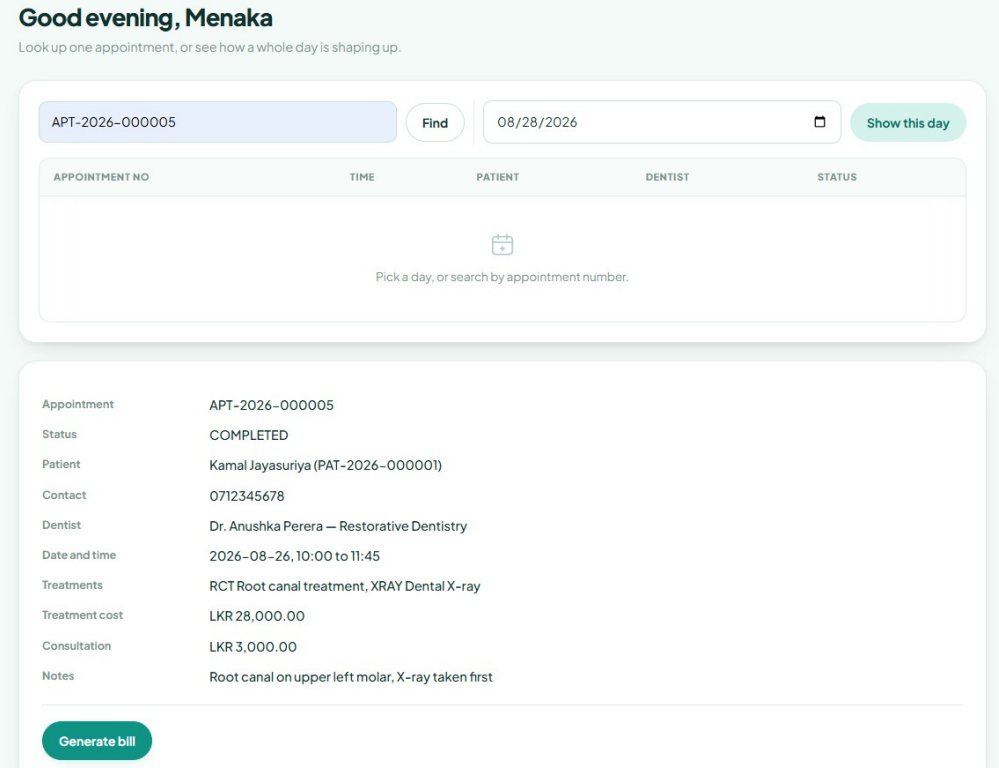


Figure 15: UC-10 search and display.

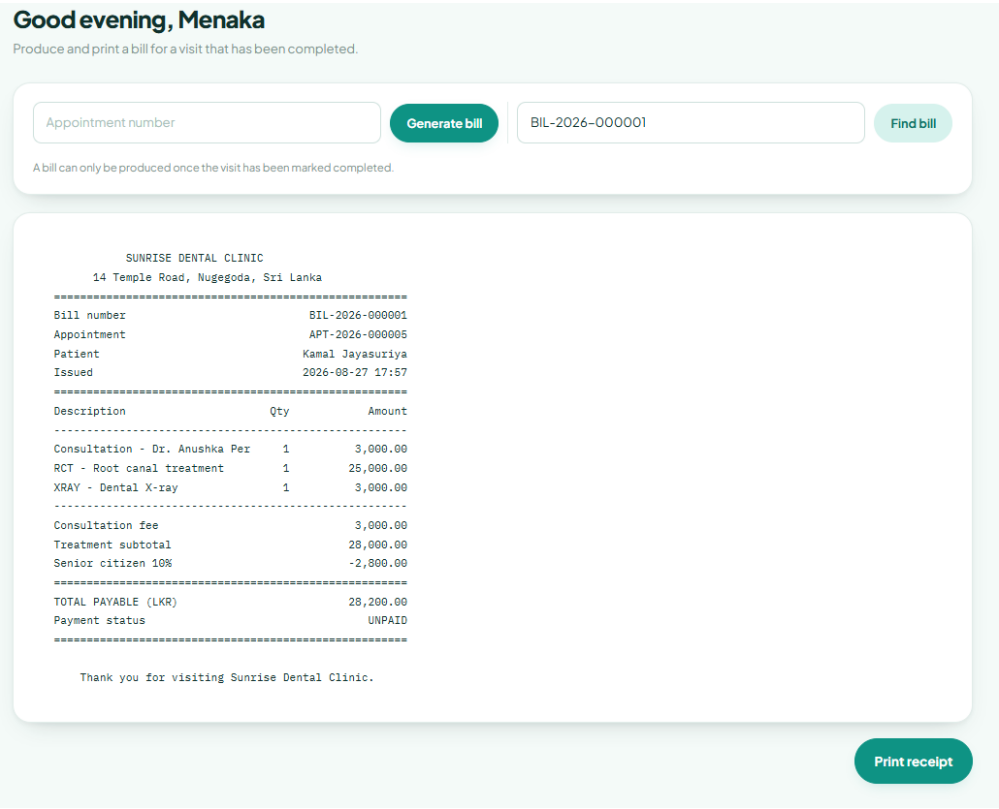


Figure 16: Bill with the discount line chosen by the Strategy.

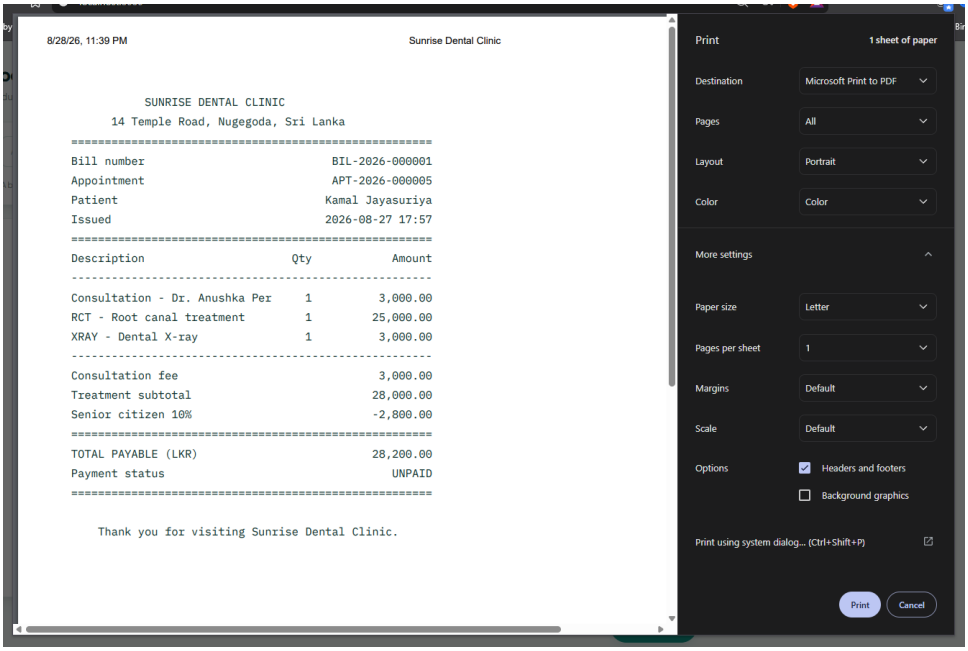


Figure 17: FR-19: the print stylesheet leaves only the receipt.

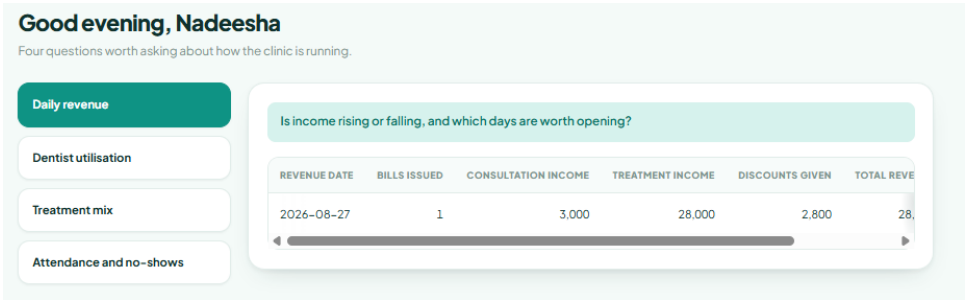


Figure 18: One of five management report views.

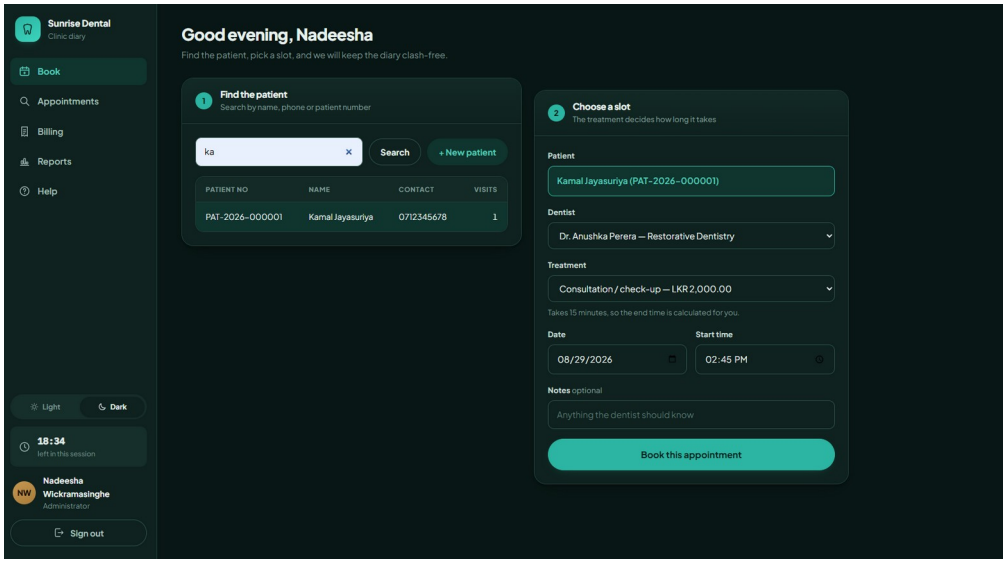


Figure 19: Light, dark and system theming.

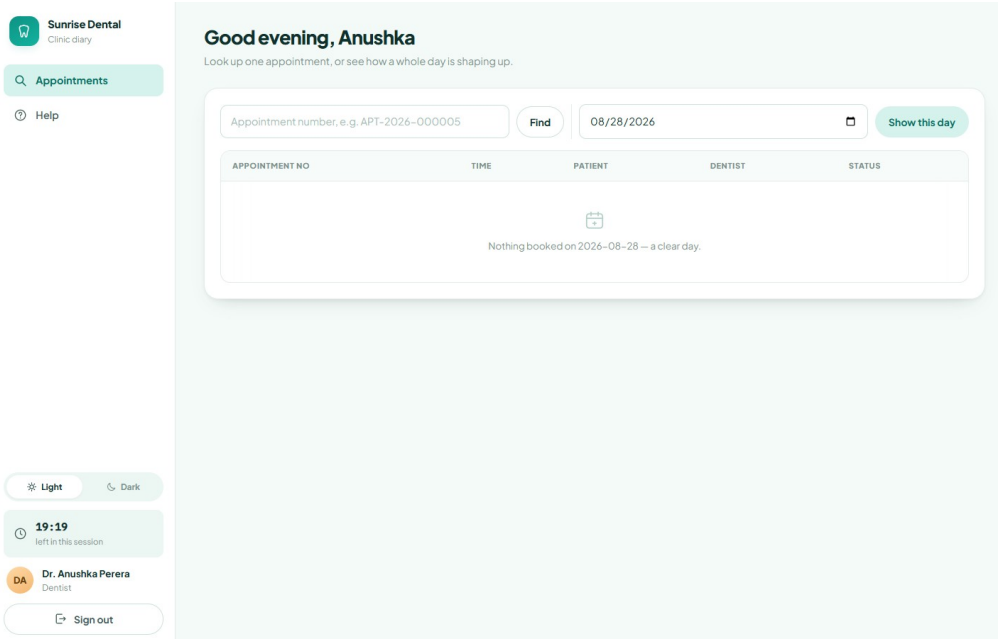


Figure 20: The same system signed in as a dentist.

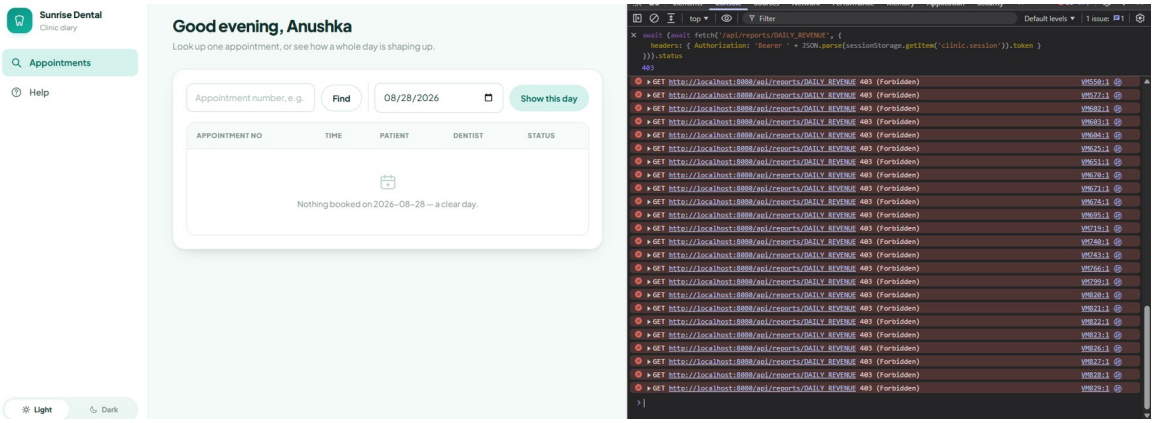


Figure 21: HTTP 403 from a dentist session: authorisation is enforced on the server.

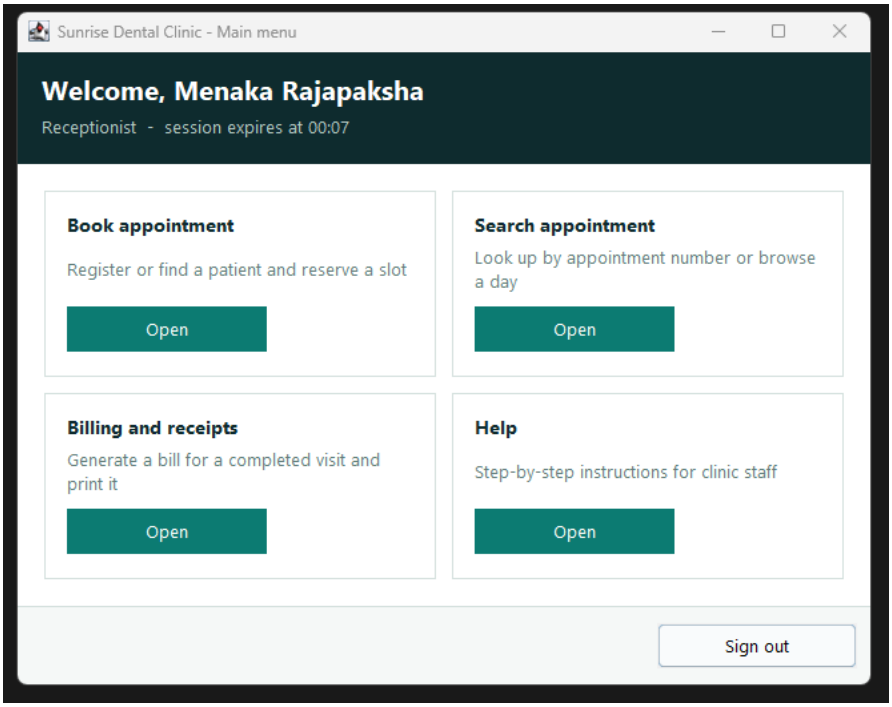


Figure 22: The Swing client, a second client over the same API.

```
2026-09-05T02:11:20.397+05:30  INFO 31340 --- [sunrise-clinic-service]
[         task-1] l.s.clinic.pattern.NotificationObserver  : Confirmati
on email sent to Thanujayahasaranga1112@gmail.com for appointment APT-2
026-000013
```

Figure 23: FR-20: the confirmation dispatched, on the async task-1 thread.

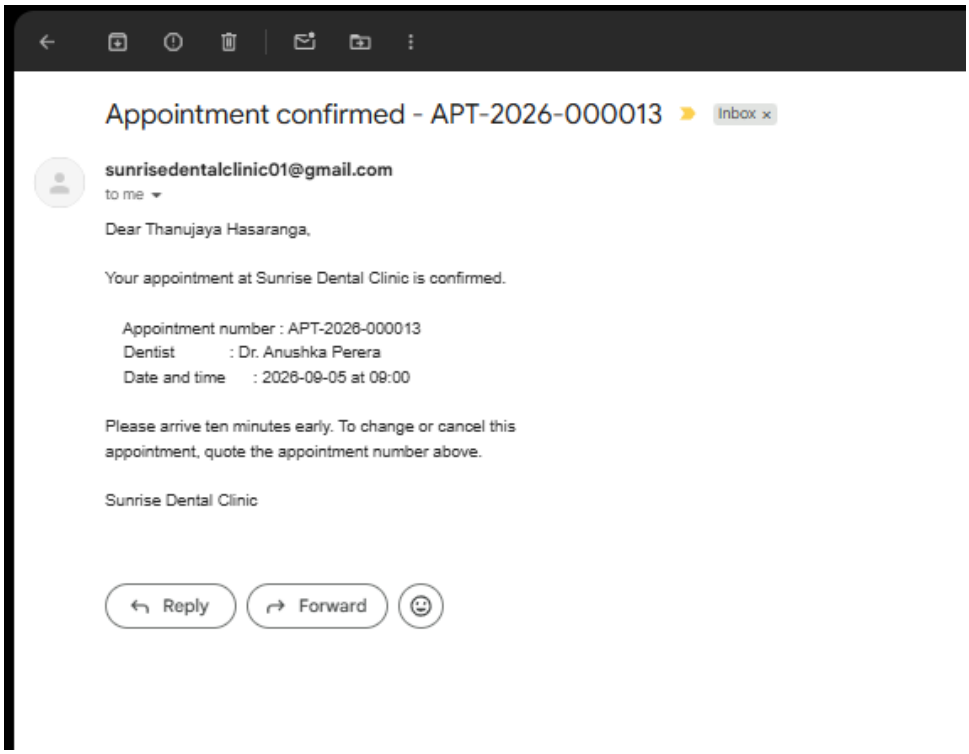


Figure 24: FR-20: the same appointment's confirmation as delivered.

6. Testing

The suite contains **99 automated tests**, all passing, run by Maven and re-run by GitHub Actions on every push (Figure 25). The exhaustive plan is Appendix A; this section gives the rationale, the derivation, the traceability and the result.

6.1 Testing at the level that can prove the thing

Each tier is tested by the technique that can prove it, and nothing twice. Pure rules are tested with no collaborators; the business tier with the repository replaced by a test double; the rules living in MySQL against a real MySQL 8 server, because the point of a trigger is that it holds *inside the transaction*; the presentation tier manually, with recorded evidence.

6.2 Test-driven development

Two features were written test-first, with red and green as separate commits so the sequence is visible in the history rather than claimed: `bb1ac3d` → `94d7a0b` for the clash rule, `ffa86d5` → `8b73b14` for the discount strategies (Figure 29). Checking out a red commit and running `mvn test` reproduces the failure; the green commit makes the same tests pass without altering them.

TDD changed the design, which is its value rather than the tests it leaves behind (Beck, 2003). Writing `AppointmentOverlapTest` first forced the clash rule out of the service and onto `Appointment.overlapsWith()`, where it runs without a database. Writing `DiscountStrategyTest` first made the four rules separate classes, because a branch of an `if`/`else` chain cannot be tested in isolation.

6.3 Deriving the test data

Test data was derived from the specification by two standard techniques rather than invented (Myers *et al.*, 2011).

Equivalence partitioning divides each input into classes the system should treat identically and takes one value from each; testing 09:00 and 09:15 proves nothing new because they fall in the same class. Partitions for the booking request and for credentials are tabulated in Appendix A.

Boundary value analysis then tests each edge at the value itself and at the adjacent value either side, because every failure the clinic reported happens at an edge.

Rule	Just below	On the boundary	Just above
Opening time 08:00	07:45 refused	**08:00 accepted**	08:15 accepted
Closing 20:00, 30-min treatment	19:15 accepted	**19:30 accepted** (ends 20:00)	19:45 refused
10-min buffer vs a 10:00–11:30 booking	09:15 accepted	**09:30 refused**	11:45 accepted
Senior discount, age 65	64 → none	**65 → 10%**	66 → 10%
Loyalty, fifth visit	3 previous → none	**4 previous → 5%**	5 previous → 5%

6.4 A hand-written test double rather than a mocking framework

`spring-boot-starter-test` supplies Mockito, so mocks were available. `InMemoryClinicRepository` was written by hand instead. A mock verifies that a **method was called**; the double holds state, so a test asserts on the **consequence** — the appointment now exists, the status really changed, the audit trail records it. Fowler (2007) frames this as behaviour versus state verification: the former couples a test to *how* the code is written, so refactoring breaks tests that should not care. Meszaros (2007) adds that a readable fake documents the contract, which fifteen `when(...).thenReturn(...)` lines do not.

The double deliberately does **not** re-implement the overlap trigger or `sp_generate_bill`. Re-implementing them in Java would only test the copy; those rules are proved against a real server instead (Figures 9 and 10).

6.5 Traceability — how each requirement is met and proved

Requirements	Met by	Proved by
FR-01…05 sign-in, lockout, session, roles	`AuthenticationService`, BCrypt, `staff.locked`	UT-AUTH-01…20; Figs 11, 21
FR-06, 07 register and find a patient	`patient`, `trg_patient_number`	DB-01; UT-APPT-09; MAN-03
FR-08, 09 numbering and booking	`trg_appointment_number`, `book()`	UT-APPT-01, 02; DB-04
FR-10, 12 clash rule and opening hours	`overlapsWith()`, `assertWithinOpeningHours()`, `trg_appointment_no_overlap`	UT-DOM-01…12; UT-APPT-11…27; DB-02, 03; Fig 9

Requirements	Met by	Proved by
FR-11 alternative slots	`nextFreeSlots()`	UT-APPT-23; Fig 14
FR-13, 14 search and display	`vw_appointment_detail`	UT-APPT-34; Fig 15; MAN-05
FR-15, 16 cancel, complete, no-show	`cancel()`, `markCompleted()`, `trg_appointment_overlap_upd`	UT-APPT-29…33
FR-17…19 billing and receipt	`BillingFacade`, `sp_generate_bill`, `@media print`	UT-BILL-01…15; DB-05, 06; Figs 10, 16, 17
FR-20, 24 notification and audit	`NotificationObserver` (SMTP), `AuditLogObserver`	UT-APPT-03, 04; UT-AUTH-03, 14; Figs 23, 24
FR-21…23 maintenance and reports	`treatment_type`, `staff`, `dentist`, five views	UT-APPT-10; DB-01; Fig 18; MAN-06, 08, 09
FR-25, 26 help and safe exit	help panel, `logout()`	UT-AUTH-15; MAN-10, 11
NFR-01, 05 tiers and concurrency	HTTP boundary; service check **and** trigger	suite runs with no database; UT-APPT-28; DB-02
NFR-02 patterns, all three families	`pattern/` package	UT-BILL-05…11; UT-PAT-01…18
NFR-03, 08 schema and search speed	3NF schema with indexes	DB-01; Fig 7; MAN-12
NFR-04, 06 validation and data restriction	service validation, `requireRole`, audit by reference	UT-APPT-05…08; UT-AUTH-07, 19; Fig 21
NFR-07, 09 automation and usability	`ci.yml`; browser UI	Figs 27, 28; MAN-13; Figs 11–14

Every requirement traces forward to the class or database object that meets it and to the test or figure that proves it. Per-requirement rows, the full 99-test plan and the manual test cases are in Appendix A.

6.6 What the testing found, and coverage read honestly

Six defects are recorded in Appendix A; two were found by testing rather than review. A boundary case was first written at 09:35, off the fifteen-minute grid, so it failed validation *before* reaching the clash rule — the test passed while testing nothing it claimed. More seriously, `tokenExpires` set the session window to zero minutes, stamping the expiry at exactly "now"; whether the check saw that as past depended on the platform clock, so it passed on Linux and in CI and failed intermittently on Windows. The fix sets a window that has already elapsed. `Thread.sleep` was rejected:

it hides the race behind a delay and slows every future run. A test whose result depends on clock granularity is worse than no test, because it teaches a team to ignore red builds.

Line coverage is 47.5% overall and branch coverage 73.8% (Figure 26). The headline is the least interesting number available, because coverage measures what was *executed*, not what was *proved*. Two things matter more: branch coverage where the decisions are — `service` 91.3%, `pattern` 94.4% — and where the uncovered code sits. It sits in `repository` (5%) and `api` (0%), which is correct: one is SQL, proved against real MySQL instead; the other delegates rather than decides, so covering it would test Spring's request mapping. Raising the headline honestly would mean `@WebMvcTest` slice tests, recorded in §8 as future work rather than added as padding.

```
Results:

Tests run: 99, Failures: 0, Errors: 0, Skipped: 0


--- jacoco:0.8.12:report (report) @ clinic-service ---
Loading execution data file C:\Users\thanu\Documents\sunrise-dental-clinic\clinic-service\target\jacoco.exec
Analyzed bundle 'Sunrise Dental Clinic Service' with 45 classes
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  4.895 s
Finished at: 2026-09-03T23:05:21+05:30
------------------------------------------------------------------------
```

Figure 25: 99 tests run, 0 failures, 0 errors.

Sunrise Dental Clinic Service

Sunrise Dental Clinic Service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
lk.sunrise.clinic.repository		5%		20%	47	52	126	135	32	37	2	3
lk.sunrise.clinic.api		0%		0%	35	35	76	76	32	32	4	4
lk.sunrise.clinic.service		87%		91%	18	82	25	196	10	36	2	6
lk.sunrise.clinic.domain		55%		77%	33	48	40	73	29	37	1	4
lk.sunrise.clinic.pattern		65%		94%	8	29	13	42	7	19	2	8
lk.sunrise.clinic.dto		79%		n/a	4	11	4	11	4	11	4	11
lk.sunrise.clinic		0%		n/a	2	2	3	3	2	2	1	1
lk.sunrise.clinic.exception		100%		n/a	0	9	0	11	0	9	0	8
Total	1,565 of 2,998	47%	44 of 168	73%	147	268	287	547	116	183	16	45

Figure 26: JaCoCo coverage by package.

7. Version control, continuous integration and deployment

The repository is public at `github.com/Thanujaya448/sunrise-dental-clinic`.

7.1 Branching and history

Work was done on short-lived `feature/` branches, each merged into `main` through a pull request rather than committed directly (Figure 30). Commit messages state what changed and why, and the two test-driven cycles appear as separate red and green commits so the sequence can be verified rather than taken on trust (Figure 29).

7.2 Continuous integration

`.github/workflows/ci.yml` runs on every push and every pull request. It has three jobs, one per tier, and a failure in any blocks the merge (Figure 25).

`unit-tests` builds on JDK 17, runs the 99 tests with JaCoCo and uploads the JUnit reports, the coverage report and the jar as artefacts. A per-package coverage table is written to the run summary, so a reviewer sees the figures without downloading anything.

`database-rules` is the job worth examining (Figure 26). GitHub starts a real MySQL 8 container, builds the schema from the five SQL scripts, runs the fourteen-row health check and fails if any row reads `FAIL`. It then asserts the business rules directly: a clashing appointment must be **rejected**, a booking inside the ten-minute buffer must be **rejected**, a legal booking must be accepted, `sp_generate_bill` must total 28,200.00, and a second bill for the same appointment must be refused. Two of those steps pass only when MySQL returns an error — the shell inverts the exit status, so a silent acceptance fails the build.

This is the strongest evidence in the submission: a machine that had never seen the database built it from scratch and confirmed the trigger enforces the clinic's core rule. It also proves the scripts are complete and ordered, which no local run can, because a local database accumulates state.

`desktop-client` packages the Swing client, so the second client cannot silently break against the shared API contract.

The pipeline earned its place immediately: the flaky session-expiry test in §6.6 was green on Linux and in CI and red on Windows, and running the same commit in more than one environment is what exposed it (Humble and Farley, 2010).

7.3 Deployment

`mvn verify` produces an executable jar with an embedded Tomcat, released under tag `v1.0` (Figure 31). It runs with:

```
java -jar clinic-service-0.1.0-SNAPSHOT.jar --
spring.datasource.password=... ^
     --clinic.notifications.email.enabled=true --
spring.mail.username=...
```

The database password and the mail credentials are supplied at runtime, never committed. The repository holds placeholders and CI uses a throwaway credential for its disposable container. A committed password cannot be un-committed from a public history.

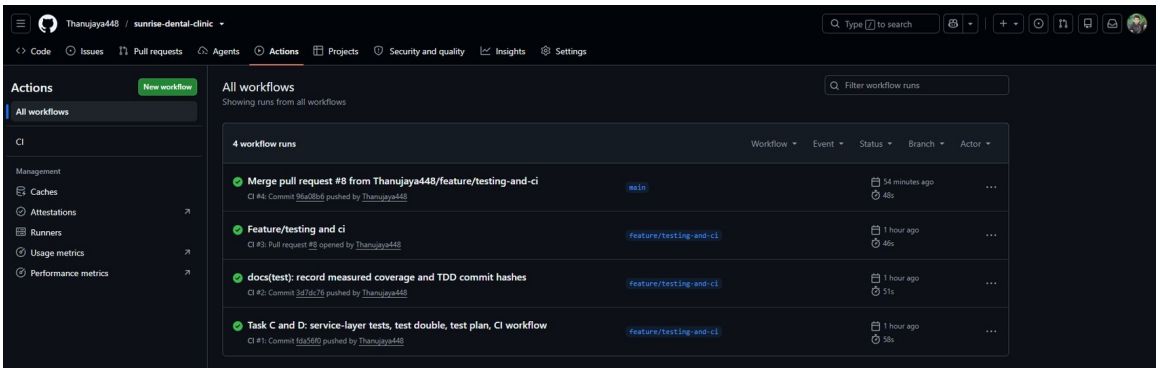


Figure 27: Every CI run green, on the branch and on main.

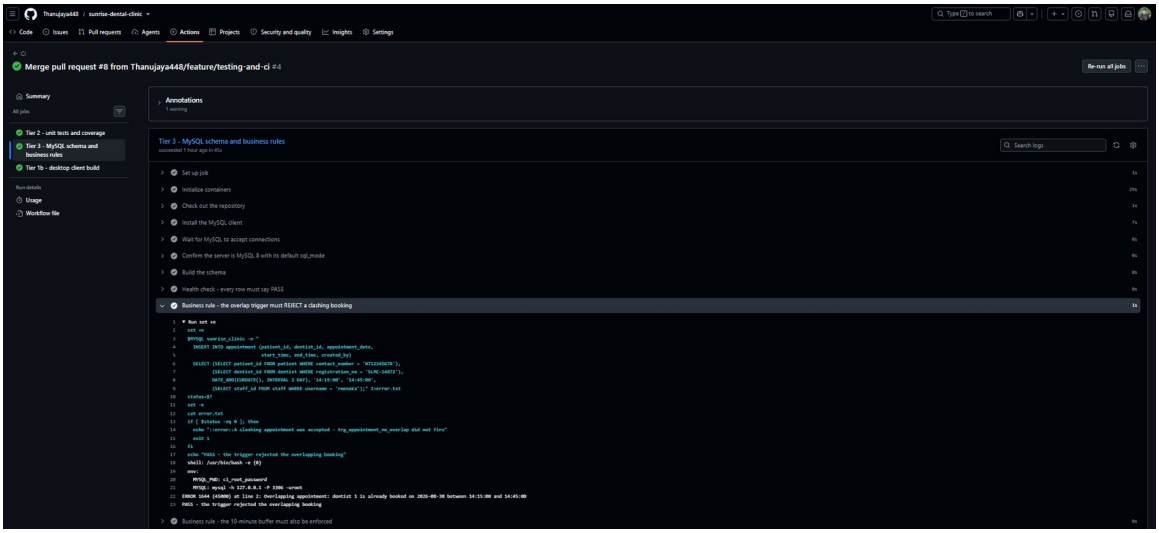


Figure 28: The database-rules job: a fresh MySQL 8 confirming the trigger fires.

```
C:\Users\thanu\Documents\sunrise-dental-clinic>git log --oneline
3d7dc76 (HEAD -> feature/testing-and-ci, origin/feature/testing-and-ci) docs(test): record measured coverage and TDD commit hashes
fda56f0 Task C and D: service-layer tests, test double, test plan, CI workflow
447ae66 (feat/web-client) feat(web): add dark mode with system-preference default and per-device override
d1642e6 fix(web): hidden attribute overridden by grid layout; feat(web): sidebar layout, day statistics, scroll affordances
21d2135 (origin/feat/web-client) feat(web): browser client with session handling; docs(uml): add browser tier to design diagram
576d837 feat(web): browser client over the same REST API with session handling
527ab4a (origin/feat/database-scripts, feat/database-scripts) feat(db): schema, triggers, routines, views and seed data; feat(api): treatments endpoint
67ca8da feat(db): add schema, triggers, routines, seed data, views and ERD
907f764 feat(db): add schema, triggers, routines, seed data, views and ERD
990d9d4 (origin/main, origin/HEAD, main) Merge pull request #5 from Thanujaya448/feat/discount-strategy
8b73b14 (origin/feat/discount-strategy, feat/discount-strategy) feat(pattern): implement discount strategies with BigDecimal rounding
ffa86d5 test(pattern): failing tests for discount strategies (FR-17, ASM-12)
9223741 test(pattern): failing tests for discount strategies (FR-17, ASM-12)
8d019ee Merge pull request #4 from Thanujaya448/feat/appointment-clash-rule
94d7a0b (origin/feat/appointment-clash-rule, feat/appointment-clash-rule) feat(domain): implement clash rule with turnaround buffer
bb1ac3d test(domain): failing tests for appointment clash rule (FR-10, ASM-08)
d202ea3 Merge pull request #3 from Thanujaya448/docs/uml-sequence-diagrams
2d3b3da (origin/docs/uml-sequence-diagrams, docs/uml-sequence-diagrams) docs(uml): add sequence diagrams; revise design classes for per-service proxies
b6afe31 Merge pull request #2 from Thanujaya448/docs/uml-diagrams
0085954 (origin/docs/uml-diagrams, docs/uml-diagrams) docs(uml): add use case and class diagrams with PlantUML sources
448e878 Merge pull request #1 from Thanujaya448/docs/initial-readme
a04cb7d (origin/docs/initial-readme, docs/initial-readme) docs: describe scenario, architecture and tech stack
71e6b52 Update README with project description
f00ddc6 Initial commit
```

Figure 29: The two test-first cycles as red and green commit pairs.

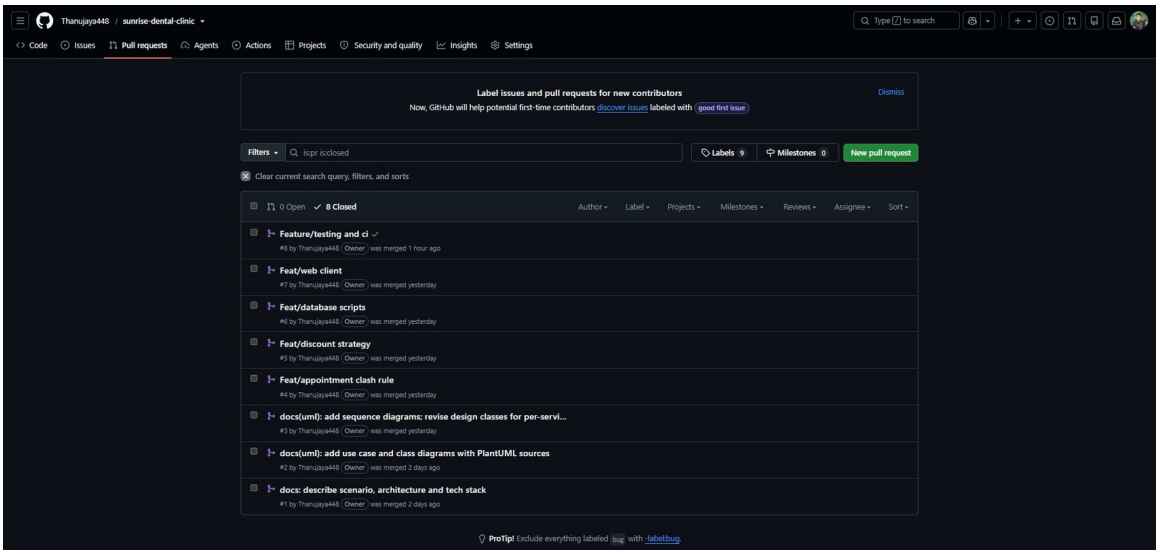


Figure 30: Merged pull requests; nothing committed direct to main.

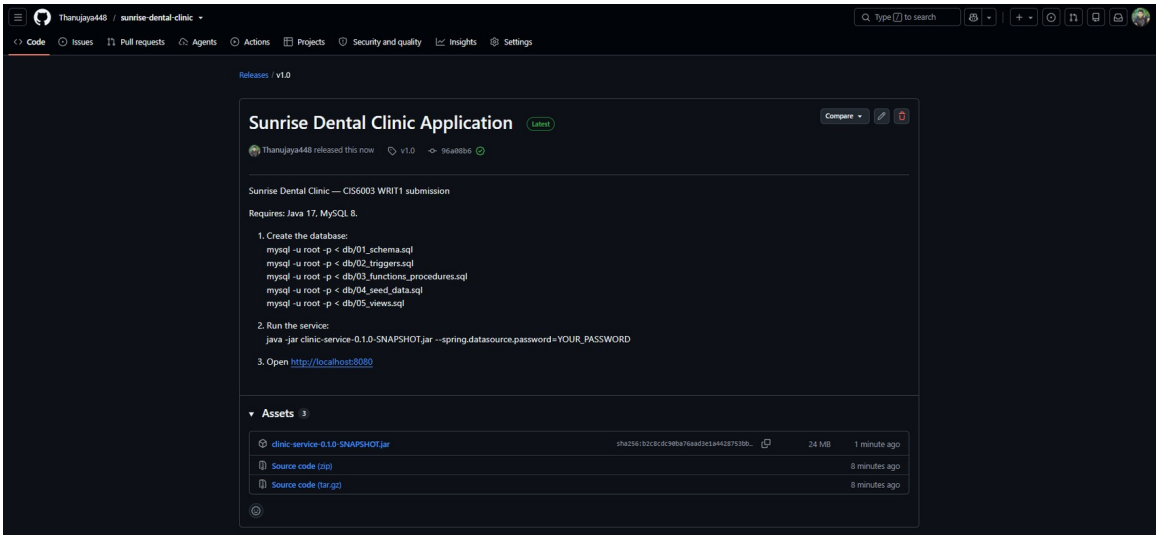


Figure 31: The tagged v1.0 release with the executable jar.

```
==================================================
 Sunrise Clinic service started on port 8080
 MySQL connected - 10 active treatment types loaded
 Try: http://localhost:8080/api/treatments
==================================================
```

Figure 32: The service starting and connecting to MySQL.

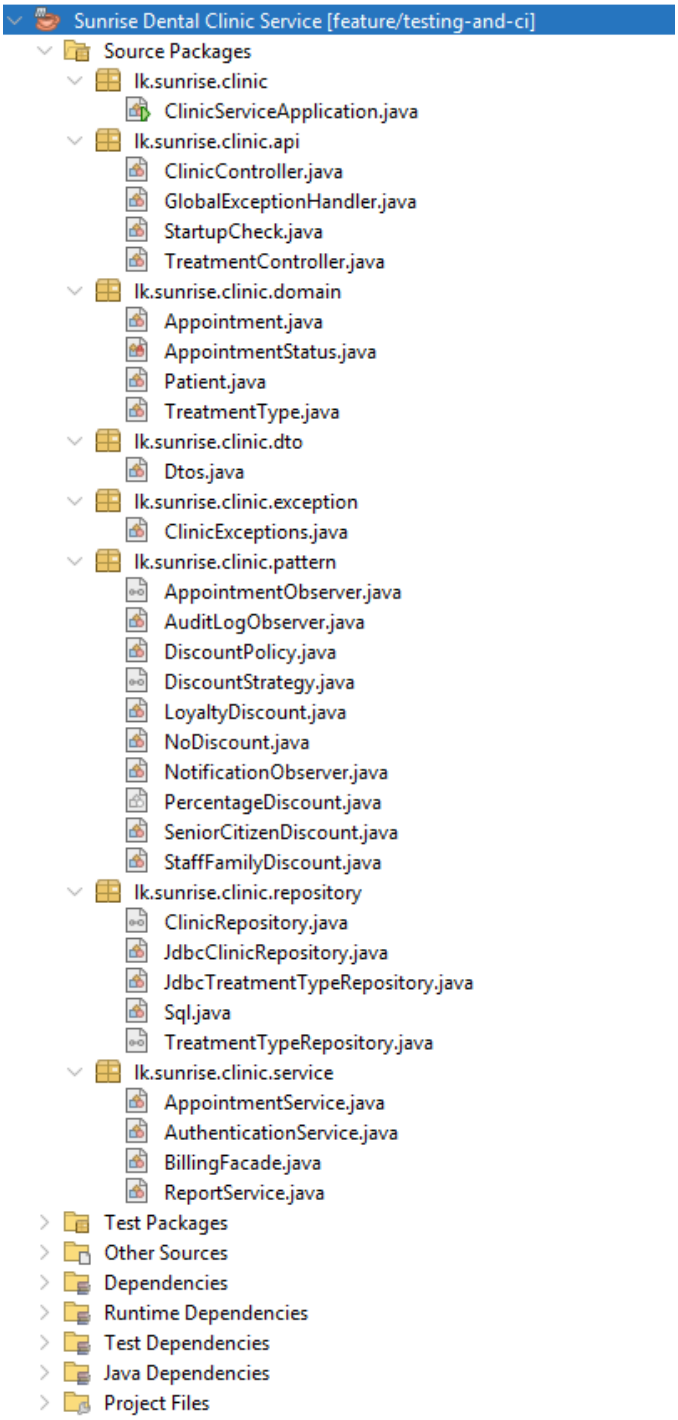


Figure 33: Project structure in Apache NetBeans.

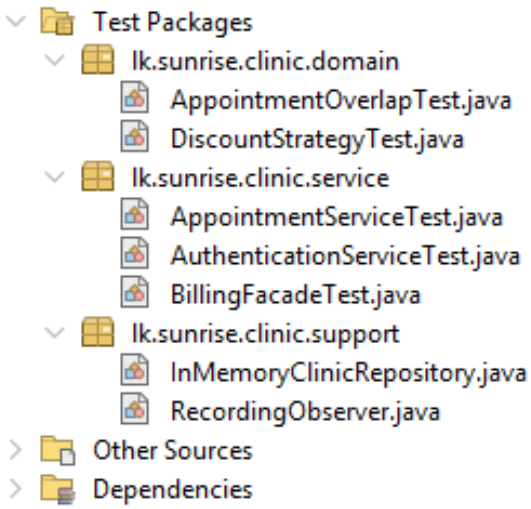


Figure 34: The five test classes and the test doubles.

8. Critical evaluation, limitations and future work

All four failures the clinic described are addressed: double bookings are prevented at three points, the last inside the database transaction; patient records persist and are referenced rather than copied; a refused booking offers alternatives; and the bill total is computed once, in one transaction, and read back rather than recalculated. What follows is what the system does not do.

A promise the system cannot keep. After five failed sign-ins an account locks and the message says to ask the Administrator to unlock it. There is no unlock function — no endpoint, no screen. `resetLoginFailures` clears the counter on a successful sign-in but never the lock, so the only route back is a manual `UPDATE`. Found while testing, this is the most serious defect remaining: a system that tells a user to do something impossible is worse than one that simply refuses. The fix is a repository method, a role-guarded endpoint and a button.

Sessions do not survive anything. Held in a `ConcurrentHashMap`, they are lost on restart and a second instance would not recognise the first's tokens. Acceptable for one clinic on one server; Redis or a signed JWT if it were not.

Suggested slots are the wrong ones. `nextFreeSlots` scans from opening time and returns the day's earliest free slots, so a receptionist asking for 14:45 is offered 08:00. It satisfies FR-11 literally and misses its intent, which was to reduce waiting. Ranking candidates by distance from the requested time would fix it.

Notification is one-directional and email-only. Confirmations are sent; cancellations are logged, because the observer contract does not carry the patient's address on that path. No SMS gateway is integrated, and no tax is modelled.

Testing has a shaped gap. Controllers sit at 0% coverage and the repository at 5%; defensible, but `@WebMvcTest` slice tests would close it honestly. The sub-two-second search requirement was verified against seeded data only, not at 10,000 records.

What I would do differently. Two process failures, both caught late. The design class diagram was drawn as intent and left to drift while the code changed, naming five classes that were never built; regenerating diagrams from the implementation at each merge would have caught that in a pull request rather than a week before submission. And a development database password reached an early commit before the runtime-

configuration approach described in §7.3 was adopted. The repository now holds only placeholders and that password has been rotated, but the real lesson is that secret scanning belongs in the pipeline from the first commit, not the last: a credential cannot be un-committed from a public history, only invalidated.

References

(Harvard. Not counted towards the 4,000 words.)

Beck, K. (2003) *Test-Driven Development: By Example*. Boston, MA: Addison-Wesley.

Codd, E.F. (1970) 'A relational model of data for large shared data banks', *Communications of the ACM*, 13(6), pp. 377–387.

Date, C.J. (2003) *An Introduction to Database Systems*. 8th edn. Boston, MA: Addison-Wesley.

Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley.

Fowler, M. (2003) *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd edn. Boston, MA: Addison-Wesley.

Fowler, M. (2007) *Mocks Aren't Stubs*. Available at: https://martinfowler.com/articles/mocksArentStubs.html (Accessed: 3 September 2026).

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Humble, J. and Farley, D. (2010) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA: Addison-Wesley.

Larman, C. (2004) *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. 3rd edn. Upper Saddle River, NJ: Prentice Hall.

Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA: Prentice Hall.

Meszaros, G. (2007) *xUnit Test Patterns: Refactoring Test Code*. Upper Saddle River, NJ: Addison-Wesley.

Myers, G.J., Sandler, C. and Badgett, T. (2011) *The Art of Software Testing*. 3rd edn. Hoboken, NJ: Wiley.

Oracle (2024) *MySQL 8.0 Reference Manual*. Available at: https://dev.mysql.com/doc/refman/8.0/en/ (Accessed: 3 September 2026).

VMware (2024) *Spring Framework Reference Documentation*. Available at: https://docs.spring.io/spring-framework/reference/ (Accessed: 3 September 2026).

Appendix A — Test plan and traceability matrix

The complete test plan, derived test data, traceability matrix, data-tier test cases and manual test cases are maintained as `docs/test-plan.md` in the repository and reproduced below.

A.1. Testing strategy

Testing follows the shape of the architecture. Each tier is proved by the technique that can actually prove it, and nothing is proved twice.

Level	What it covers	Technique	Where it lives
Unit — domain	Rules that are pure arithmetic or pure logic: the clash rule, the discount rules	JUnit 5, no collaborators at all	`domain/AppointmentOverlapTest`, `pattern/DiscountStrategyTest`
Unit — service	Tier 2 business logic: validation, scheduling, authorisation, discount selection	JUnit 5 with a hand-written **test double** replacing the repository	`service/*Test`
Integration — data tier	Rules enforced by the database: the overlap trigger, `sp_generate_bill`	SQL scripts run against a real MySQL 8 server	`db/06_verify.sql`, `db/08_healthcheck.sql`, and the `database-rules` job in CI
Manual — presentation	The browser and desktop clients end to end	Scripted manual test cases with recorded evidence	Section 7 of this document

A.1.1 Why a hand-written test double rather than Mockito

`spring-boot-starter-test` brings Mockito, so a mock was available. A hand-written `InMemoryClinicRepository` was chosen instead:

- A mock verifies that a **method was called**. The double holds state, so a test asserts on the **consequence** — the appointment now exists, the status really changed, the audit trail records it. Behaviour verification couples a test to *how* the service is written; state verification couples it only to *what* it achieves, so these tests survive refactoring.
- Fifteen `when(...).thenReturn(...)` lines do not document a contract. One readable class does.

- The double is itself the proof that the tiers are separated. The entire service layer runs in it with **no MySQL, no JDBC driver and no Spring context**. If a single business rule had leaked into a SQL string, these tests could not exist.

A.1.2 What the service tests deliberately do NOT do

The double does **not** re-implement the overlap trigger or `sp_generate_bill`. Those are data-tier rules; re-implementing them in Java would only test the copy. They are proved against a real MySQL 8 server instead — see section 6.

`BillingFacadeTest` therefore asserts that the facade selects the right **Strategy** and passes the right discount **amount** to the procedure. The arithmetic that turns that amount into a bill total is asserted in SQL.

A.1.3 Test-driven development

Two features were written test-first, and the red and green steps are separate commits so the sequence is visible in the Git history rather than merely claimed:

Cycle	Feature	Red commit — test written first, fails	Green commit — implementation, test passes
1	`Appointment.overlapsWith()` — the clash rule (FR-10, ASM-08)	`bb1ac3d` *test(domain): failing tests for appointment clash rule*	`94d7a0b` *feat(domain): implement clash rule with turnaround buffer* — 12 tests pass
2	`DiscountStrategy` family — the discount rules (FR-17, ASM-12)	`ffa86d5` *test(pattern): failing tests for discount strategies*	`8b73b14` *feat(pattern): implement discount strategies with BigDecimal rounding* — 18 tests pass

Checking out the red commit and running `mvn test` reproduces the failure; the green commit that follows it makes the same tests pass without altering them. That sequence in the history is the evidence — the tests were not written after the code to fit it.

A.2. How test data was derived

Test data is **derived from the specification**, not invented. Two techniques were applied to every rule that takes a value.

A.2.1 Equivalence partitioning

Each input is divided into classes that the system should treat identically; one value is taken from each class. Testing 09:00 and 09:15 proves nothing new, because they are in the same class.

Input	Partitions	Representative values
Booking request completeness	complete · missing patient · missing dentist · missing treatment · missing date/time	one case each
Patient number	exists · does not exist	`PAT-2026-000001` · `PAT-2026-999999`
Treatment code	active · unknown	`SCA` · `ZZZ`
Appointment day	open day · closed day (Sunday) · past date	next Monday · next Sunday · yesterday
Requested slot	free · overlapping · inside buffer · adjacent, clear	see 2.2
Appointment status when billing	`COMPLETED` · `BOOKED` · `CANCELLED` · already billed	one case each
Patient discount category	none · senior · loyalty · staff family · several at once	age 40 / 65 / 4 visits / staff flag / all three
Credentials	correct · wrong password · unknown user · locked · deactivated · blank	one case each

A.2.2 Boundary value analysis

Every defect the clinic reported happens at an edge, never in the middle of a range. Each boundary is tested at the value itself and at the adjacent value on each side.

Rule	Boundary	Just below	On the boundary	Just above
Opening time 08:00	08:00	07:45 → refused	08:00 → **accepted**	08:15 → accepted
Closing time 20:00 (30-min treatment)	19:30 start	19:15 → accepted	19:30 → **accepted** (ends 20:00)	19:45 → refused
Slot grain 15 min	multiples of 15	08:05, 08:10, 08:20 → refused	08:00, 08:15, 08:30, 08:45 → accepted	—
Turnaround buffer 10 min, existing 10:00–11:30	09:15–09:45 → accepted (widens to 09:55)	—	09:30–10:00 → **refused** (widens to 10:10)	11:45 → accepted
Senior citizen age 65	64 → no discount	**65 → 10%**	66 → 10%	
Loyalty, 5th visit	3 previous → none	**4 previous → 5%**	5 previous → 5%	

Rule	Boundary	Just below	On the boundary	Just above
Login attempts, max 5	1st failure → "4 attempts remaining"	—	—	
Session, 20 min idle	live token → accepted	0-minute window → **expired**	—	

A.2.3 Rounding

Money is `BigDecimal` with `RoundingMode.HALF_UP` to two decimal places, matching `DECIMAL(10,2)` in MySQL, and is asserted at a value that actually rounds — a subtotal chosen so that the percentage does not divide exactly. `double` is never used for money anywhere in the system.

A.3. Automated test plan — tier 2 and the domain

Every row below is one automated test. Run them with:

```
cd clinic-service
mvn -B verify
```

Measured result: **99 tests run, 0 failures, 0 errors**, JaCoCo report produced and the service JAR packaged. Screenshot the Maven output and the JaCoCo summary page into `docs/evidence/`.

A.3.1 Authentication — AuthenticationServiceTest (20 tests)

Test ID	Requirement	Test method	Input / condition	Expected outcome
UT-AUTH-01	FR-01	`SignIn.signsInSuccessfully`	`reception1` + correct password	Token issued; role `RECEPTIONIST`; expiry in the future
UT-AUTH-02	FR-02	`SignIn.storedValueIsAHashNotThePassword`	Stored credential inspected	Value starts `$2`, is 60 chars, never contains the password
UT-AUTH-03	FR-24	`SignIn.writesAuditEntry`	Successful sign-in	A `LOGIN` row is written to the audit trail
UT-AUTH-04	FR-03	`SignIn.resetsFailureCounter`	Successful sign-in after failures	Failure counter reset for that staff ID
UT-AUTH-05	NFR-04	`SignIn.refusesBlankInput`	`""`, `"  "`, `null`	`ValidationException` before

Test ID	Requirement	Test method	Input / condition	Expected outcome
				the database is touched
UT-AUTH-06	FR-03	`Refusal.wrongPasswordIsCounted`	Correct user, wrong password	`AuthenticationFailedException`; failure recorded once
UT-AUTH-07	NFR-06	`Refusal.doesNotRevealWhichFieldWasWrong`	Unknown username	Message is exactly the generic "Invalid username or password" — no user enumeration
UT-AUTH-08	FR-03	`Refusal.countsDownRemainingAttempts`	1st failure of 5 allowed	Message contains "4 attempts remaining"
UT-AUTH-09	FR-03	`Refusal.lockedAccountIsRefusedDistinctly`	Locked account, correct password	`AccountLockedException`, message names the lock
UT-AUTH-10	FR-05	`Refusal.deactivatedAccountIsRefused`	Inactive account	Generic authentication failure
UT-AUTH-11	FR-04	`Sessions.tokenResolvesToSession`	Fresh token	Resolves to the correct username and staff ID
UT-AUTH-12	FR-04	`Sessions.rejectsMissingAndUnknownTokens`	`null`, blank, forged UUID	`SessionExpiredException` in all three cases
UT-AUTH-13	FR-04	`Sessions.tokenExpires`	`SESSION_MINUTES = -1`, i.e. a window that has already elapsed	`SessionExpiredException`, message says "expired"
UT-AUTH-14	FR-04	`Sessions.logoutInvalidatesToken`	Sign out, then reuse the token	Refused; `LOGOUT` written to the audit trail
UT-AUTH-15	FR-26	`Sessions.logoutWithoutTokenDoesNotThrow`	`logout(null)`	Returns quietly — a safe exit never crashes
UT-AUTH-16	FR-04	`Sessions.tokensAreUnique`	Two different sign-ins	Two different tokens
UT-AUTH-17	FR-05	`RoleChecks.allowsPermittedRole`	Receptionist, receptionist-or-admin action	Allowed
UT-AUTH-18	FR-05	`RoleChecks.refusesWrongRole`	Dentist, receptionist-only action	`ForbiddenException`
UT-AUTH-19	FR-05	`RoleChecks.refusesWithoutAnyToken`	No token; forged token	`SessionExpiredException` — hiding a button in the client is not a control

Test ID	Requirement	Test method	Input / condition	Expected outcome
UT-AUTH-20	FR-05	`RoleChecks.allowsAdministrator`	Administrator, admin-only action	Allowed, role returned

A.3.2 Appointments — AppointmentServiceTest (34 tests)

Test ID	Requirement	Test method	Input / condition	Expected outcome
UT-APPT-01	FR-09, FR-08	`HappyPath.booksSuccessfully`	Valid request, free slot	Appointment stored, status `BOOKED`, number `APT-YYYY-NNNNNN`
UT-APPT-02	FR-09, ASM-07	`HappyPath.derivesEndTimeFromTreatmentDuration`	30-min scaling; 90-min root canal	Ends 08:30 and 15:30 respectively — duration comes from the treatment, not the user
UT-APPT-03	FR-20	`HappyPath.publishesToObservers`	Valid booking	Observer notified exactly once, with the patient's name
UT-APPT-04	FR-20	`HappyPath.worksWithNoObservers`	No observers registered	Booking still succeeds — listeners are optional
UT-APPT-05	NFR-04	`Validation.rejectsMissingPatient`	Blank patient number	`ValidationException`
UT-APPT-06	NFR-04	`Validation.rejectsMissingDentist`	`null` dentist	`ValidationException`
UT-APPT-07	NFR-04	`Validation.rejectsMissingTreatment`	`null` treatment code	`ValidationException`
UT-APPT-08	NFR-04	`Validation.rejectsMissingDateOrTime`	Missing date; missing time	`ValidationException` in both cases
UT-APPT-09	FR-07	`Validation.rejectsUnknownPatient`	`PAT-2026-999999`	`NotFoundException` — a 404, not a 400
UT-APPT-10	FR-21	`Validation.rejectsUnknownTreatmentCode`	Code `ZZZ`	`NotFoundException`
UT-APPT-11	FR-12	`OpeningHours.openingBoundary`	08:00; 07:45	Accepted; refused
UT-APPT-12	FR-12	`OpeningHours.closingBoundary`	19:30; 19:45 (30-min treatment)	Accepted; refused

Test ID	Requirement	Test method	Input / condition	Expected outcome
UT-APPT-13	FR-12, ASM-06	`OpeningHours.rejectsClosedDay`	Next Sunday	`ValidationException` naming the closed day
UT-APPT-14	FR-12	`OpeningHours.rejectsPastDate`	Yesterday	`ValidationException`
UT-APPT-15…21	FR-12, ASM-06	`OpeningHours.enforcesSlotGranularity` (7 parameterised cases)	08:00 / 08:15 / 08:30 / 08:45 accepted; 08:05 / 08:10 / 08:20 refused	Only multiples of 15 minutes are accepted
UT-APPT-22	FR-10	`ClashDetection.refusesOverlap`	10:30 against an existing 10:00–11:30	`SlotUnavailableException`
UT-APPT-23	FR-11, ASM-09	`ClashDetection.offersAlternatives`	Same clash	Exception carries at most 3 free slots, none inside the busy period plus buffer
UT-APPT-24	FR-10, ASM-08	`ClashDetection.bufferBeforeExistingAppointment`	09:30 (widens to 10:10); 09:15 (widens to 09:55)	Refused; accepted
UT-APPT-25	FR-10, ASM-08	`ClashDetection.bufferAfterExistingAppointment`	11:45 after an 11:30 end	Accepted — clears the 10-minute turnaround
UT-APPT-26	FR-10	`ClashDetection.differentDentistDoesNotClash`	Same time, Dr. Fernando	Accepted — the rule is per dentist
UT-APPT-27	FR-10	`ClashDetection.differentDayDoesNotClash`	Same time, next day	Accepted
UT-APPT-28	NFR-05	`ClashDetection.secondIdenticalBookingIsRefused`	Same slot booked twice in a row	Second attempt refused — the first booking occupies the slot
UT-APPT-29	FR-15	`StatusTransitions.cancelsWithReason`	Cancel a booked appointment with a reason	Status `CANCELLED`; observers notified
UT-APPT-30	FR-15, FR-24	`StatusTransitions.refusesCancellationWithoutReason`	Blank reason	`ValidationException`; appointment left untouched
UT-APPT-31	FR-15	`StatusTransitions.refusesDoubleCancellation`	Cancel twice	Second attempt refused
UT-APPT-32	FR-16, ASM-05	`StatusTransit`	Complete a	`ValidationExc`

Test ID	Requirement	Test method	Input / condition	Expected outcome
		`ions.refusesCompletingACancelledAppointment`	cancelled appointment	`eption`
UT-APPT-33	FR-16	`StatusTransitions.completesBookedAppointment`	Complete a booked appointment	Status `COMPLETED` — the precondition for billing
UT-APPT-34	FR-13	`StatusTransitions.unknownAppointmentNumberIsNotFound`	`APT-2026-999999`	`NotFoundException`

A.3.3 Billing — BillingFacadeTest (15 tests)

Test ID	Requirement	Test method	Input / condition	Expected outcome
UT-BILL-01	FR-17	`Preconditions.unknownAppointment`	Unknown appointment number	`NotFoundException`
UT-BILL-02	FR-18, ASM-05	`Preconditions.refusesBookedAppointment`	Status `BOOKED`	`BillingException` naming the precondition
UT-BILL-03	FR-18, ASM-05	`Preconditions.refusesCancelledAppointment`	Status `CANCELLED`	`BillingException`
UT-BILL-04	FR-18	`Preconditions.refusesSecondBill`	Bill the same appointment twice	Second attempt refused, message says "already"
UT-BILL-05	ASM-12	`DiscountSelection.ordinaryPatientGetsNoDiscount`	Age 40, no flags, 0 visits	Amount `0.00`, label "No discount" — the **Null Object**, never `null`
UT-BILL-06	ASM-12	`DiscountSelection.seniorGetsTenPercent`	Age exactly 65, subtotal 10,000	`1000.00`, label contains "Senior"
UT-BILL-07	ASM-12	`DiscountSelection.sixtyFourIsNotSenior`	Age 64	`0.00` — the boundary is exact
UT-BILL-08	ASM-12	`DiscountSelection.fifthVisitEarnsLoyalty`	4 previous visits	`500.00`, label "Returning patient 5%"
UT-BILL-09	ASM-12	`DiscountSelection.fourthVisitEarnsNothing`	3 previous visits	`0.00`

Test ID	Requirement	Test method	Input / condition	Expected outcome
UT-BILL-10	ASM-12	`DiscountSelection.staffFamilyGetsFifteenPercent`	Staff family flag	`1500.00`
UT-BILL-11	ASM-12	`DiscountSelection.mostGenerousRuleWins`	Senior **and** loyal **and** staff family	`1500.00` only — rules do not stack
UT-BILL-12	FR-17, FR-19	`BillContents.returnsStoredBill`	Bill a completed appointment	Bill read back from the data tier; number `BIL-YYYY-NNNNNN`; status `UNPAID`
UT-BILL-13	FR-19	`BillContents.unknownBillNumber`	Unknown bill number	`NotFoundException`
UT-BILL-14	NFR-04	`DatabaseRuleSurfaced.translatesDataAccessException`	Stored procedure signals `SQLSTATE 45000`	The procedure's own business message reaches the user, not a stack trace
UT-BILL-15	NFR-04	`DatabaseRuleSurfaced.handlesMessagelessFailure`	Failure with no message	Falls back to "The bill could not be generated"

A.3.4 Domain rules — AppointmentOverlapTest (12 tests) and DiscountStrategyTest (18 tests)

Both were written **test-first** (section 1.3) and run with no collaborators at all — not even a test double.

Test ID	Requirement	Coverage
UT-DOM-01…04	FR-10, ASM-08	Overlapping ranges: identical times, contains, starts inside, ends inside
UT-DOM-05…08	ASM-08	Buffer: clears exactly, inside the buffer, clears before, zero buffer
UT-DOM-09…12	FR-10	Different key: other dentist, other day, cancelled frees the slot, later the same day
UT-PAT-01…07	ASM-12	Discount arithmetic: senior, loyalty, staff, none, rounding, zero subtotal, `null` subtotal
UT-PAT-08…13	ASM-12	Boundaries: age 64 / 65 / 66; 3 / 4 / 5 previous visits (parameterised)

Test ID	Requirement	Coverage
UT-PAT-14…18	ASM-12	Precedence: staff beats senior, senior beats loyalty, all three, none, `null` patient

A.4. Traceability matrix

Every requirement in the register traces forward to a use case, a class and the tests that prove it. Blank test cells are requirements proved by manual test or by the data tier, not by JUnit — stated rather than hidden.

Req	Description	Use case	Class(es) under test	Tests
FR-01	Staff log in	UC-01	`AuthenticationService`	UT-AUTH-01
FR-02	Passwords stored hashed	UC-01	`AuthenticationService` (BCrypt)	UT-AUTH-02, DB-08
FR-03	Lock out after 5 failures	UC-01	`AuthenticationService`	UT-AUTH-04, 06, 08, 09
FR-04	Session with idle timeout	UC-02	`AuthenticationService`	UT-AUTH-11…16
FR-05	Role-restricted functions	UC-03	`AuthenticationService`, `ClinicController`	UT-AUTH-10, 17…20
FR-06	Register a new patient	UC-05	`JdbcClinicRepository`, `patient` table	DB-01, MAN-03
FR-07	Look up an existing patient	UC-06	`AppointmentService`	UT-APPT-09, MAN-04
FR-08	Generated appointment number	UC-07	`trg_appointment_number`	UT-APPT-01, DB-04
FR-09	Book an appointment	UC-07	`AppointmentService`	UT-APPT-01, 02
FR-10	Reject a dentist overlap	UC-07	`Appointment`, `AppointmentService`, `trg_appointment_no_overlap`	UT-DOM-01…12, UT-APPT-22, 24…27, DB-02
FR-11	Suggest the next 3 free slots	UC-07	`AppointmentService.nextFreeSlots`	UT-APPT-23
FR-12	Reject out-of-hours and past dates	UC-07	`AppointmentService`	UT-APPT-11…21

Req	Description	Use case	Class(es) under test	Tests
FR-13	Search by appointment number	UC-10	`AppointmentService`, `vw_appointment_detail`	UT-APPT-34, MAN-05
FR-14	Display full appointment detail	UC-10	`vw_appointment_detail`	MAN-05
FR-15	Cancel / reschedule	UC-11	`AppointmentService`, `trg_appointment_overlap_upd`	UT-APPT-29…31
FR-16	Mark completed / no-show	UC-13	`AppointmentService`	UT-APPT-32, 33
FR-17	Calculate a bill	UC-14	`BillingFacade`, `sp_generate_bill`	UT-BILL-01, 12, DB-05
FR-18	Bill only completed appointments	UC-14	`BillingFacade`, `sp_generate_bill`	UT-BILL-02…04, DB-06
FR-19	Print a receipt	UC-15	`app.css` `@media print`, `BillingFrame`	UT-BILL-12, 13, MAN-07
FR-20	Appointment notifications	UC-08	`NotificationObserver` (**stub — logs what it would send**)	UT-APPT-03, 04
FR-21	Maintain treatments and prices	UC-16	`TreatmentTypeRepository`, `treatment_type`	UT-APPT-10, MAN-08
FR-22	Maintain staff and dentists	UC-17	`staff`, `dentist` tables	DB-01, MAN-09
FR-23	Management reports	UC-18	`ReportService`, 5 views	DB-01, MAN-06
FR-24	Audit log	UC-19	`AuditLogObserver`, `audit_entry`	UT-AUTH-03, 14, UT-APPT-30
FR-25	Help screen	UC-20	Help panel / `HelpFrame`	MAN-10
FR-26	Safe exit	UC-21	`AuthenticationService.logout`	UT-AUTH-15, MAN-11
NFR-01	Three physical tiers	—	Browser / Swing → Spring Boot :8080 → MySQL :3306	The whole service suite runs with no database at all — see 1.1

Req	Description	Use case	Class(es) under test	Tests
NFR-02	Patterns from all three families, evaluated	—	`pattern/` package	UT-BILL-05…11, UT-APPT-03, 04, UT-PAT-01…18
NFR-03	3NF schema with referential integrity	—	`01_schema.sql`	DB-01
NFR-04	Client and server validation, server authoritative	—	`AppointmentService`, `GlobalExceptionHandler`	UT-APPT-05…08, UT-BILL-14, 15, UT-AUTH-05
NFR-05	Concurrent bookings cannot both succeed	—	Service check **and** `trg_appointment_no_overlap`	UT-APPT-28, DB-02, DB-03
NFR-06	Role-restricted patient data, no PII in logs	—	`AuthenticationService`, `AuditLogObserver`	UT-AUTH-07, 19
NFR-07	Automated tests on every push	—	`.github/workflows/ci.yml`	The CI run for this commit
NFR-08	Search under 2 s at 10,000 records	—	Indexes in `01_schema.sql`	MAN-12
NFR-09	Operable by non-technical staff	—	Browser UI	MAN-13

A.5. Coverage

JaCoCo runs on every build and every CI push.

```
cd clinic-service
mvn -B verify
```

Report: `clinic-service/target/site/jacoco/index.html`. In CI it is also uploaded as the **jacoco-coverage-report** artifact, and a per-package summary is printed to the workflow run's summary page.

How to read these figures honestly. Coverage measures what was *executed*, not what was *proved*. A single test that calls every method and asserts nothing scores 100%. So the headline 47.5% is the least interesting number on this page, and quoting it alone — in either direction — would misrepresent the suite.

Two things matter more:

1. **Branch coverage on the packages that hold decisions.** A branch is an `if`, a boundary, a rule that can go either way. `service` is at **91.3%** and `pattern` at **94.4%** — nearly every decision the clinic's rules can make is exercised by a test.
2. **Where the uncovered lines are.** They are concentrated in exactly the places that *should* be uncovered by unit tests.

Measured run, `mvn -B verify`. **Note on which metric is quoted:** the table below reports **line** coverage, taken from `jacoco.csv`. The first `Cov.` column on JaCoCo's own HTML page is **instruction** coverage, which is a slightly different number for the same code — `pattern`, for example, is 65% by instruction and 69% by line. Both are given here so the figures in this document and the figures in the screenshot can be reconciled.

Package	Line cov.	Instruction cov.	Branch cov.	Reading
`lk.sunrise.clinic.service`	**87.2%**	87%	**91.3%**	The business logic. This is the figure that matters, and it is the highest in the project.
`lk.sunrise.clinic.pattern`	69.0%	65%	**94.4%**	Every discount *decision* is covered. The uncovered lines are `NotificationObserver` and `AuditLogObserver` bodies — logging, not logic.
`lk.sunrise.clinic.exception`	100%	100%	n/a	Every domain exception is thrown by a test — no dead exception types.
`lk.sunrise.clinic.domain`	45.2%	55%	77.3%	`Appointment.overlapsWith()` is fully covered; the shortfall is unused accessors on `Patient` and `TreatmentType`. Line coverage penalises getters; branch coverage shows the rule itself is proved.
`lk.sunrise.clinic.dto`	63.6%	79%	n/a	Java records — generated accessors.

Package	Line cov.	Instruction cov.	Branch cov.	Reading
`lk.sunrise.clinic.repository`	6.7%	5%	20.0%	**Deliberately low.** This layer is SQL. Executing it in a unit test would prove only that JDBC works; it is proved instead against a real MySQL 8 server in section 6.
`lk.sunrise.clinic.api`	0.0%	0%	0.0%	**Deliberately zero.** Controllers delegate and do not decide. Covering them would test Spring's request mapping, not the clinic's rules; they are proved by the manual end-to-end tests in section 7.
Overall	**47.5%**	47%	**73.8%**	Dominated by the two layers that are intentionally untested here.

Stating the gap and defending it is worth more than raising the headline figure by adding tests that assert nothing. If the number needed to rise, the honest way would be `@WebMvcTest` slice tests over the controllers — noted in the report as future work rather than added as padding.

A.6. Data-tier tests

These prove the rules that live in MySQL. They cannot be proved in Java, because the point of them is that they hold *inside the transaction*.

Run locally:

```
mysql -u root -p < db/08_healthcheck.sql
mysql -u root -p --force < db/06_verify.sql
```

`--force` is required for `06_verify.sql`: several checks are **supposed** to error, and that error is the pass condition. In CI the same assertions run as explicit shell steps in the `database-rules` job, which fails the build if a rejection does not happen.

Test ID	Requirement	What is done	Expected outcome
DB-01	NFR-03, FR-06, FR-22, FR-23	`08_healthcheck.sql`	14 rows, every one `PASS`: 12 tables, 5 views, 4 triggers, 2 routines, 9 foreign keys, seed data present
DB-02	FR-10, NFR-05	Insert 14:15–14:45 against an existing 14:00–14:30	`SQLSTATE 45000` — rejected by `trg_appointment_no_overlap`
DB-03	ASM-08, NFR-05	Insert 14:35–15:05 — no overlap, but inside the 10-minute buffer	`SQLSTATE 45000` — rejected
DB-04	FR-08, FR-09	Insert 14:45–15:15 — clears the buffer	Accepted; `appointment_no` generated as `APT-YYYY-NNNNNN`
DB-05	FR-17, ASM-10, ASM-12	`CALL sp_generate_bill` for the senior citizen: 3,000 fee + 28,000 treatments − 2,800 discount	`total_payable = 28200.00`, bill lines written in the same transaction
DB-06	FR-18	Call `sp_generate_bill` again for the same appointment	Rejected — an appointment may be billed once only
DB-07	ASM-05	Call `sp_generate_bill` for a `BOOKED` appointment	Rejected — only completed appointments are billable
DB-08	FR-02	Inspect `staff.password_hash`	All 5 rows are BCrypt hashes (`$2…`), no plain text anywhere

A.7. Manual test cases — presentation tier

Automated tests cannot prove that a receptionist can use the screen. These are run by hand against the running system, and the evidence is a screenshot of **your own** system taken at the moment of the test.

Test ID	Requirement	Steps	Expected result
MAN-01	FR-01, NFR-09	Open `http://localhost:8080`, sign in as a receptionist	Dashboard opens showing only the receptionist's tabs
MAN-02	FR-05	Sign in as a dentist	Billing and admin tabs are absent; calling the

Test ID	Requirement	Steps	Expected result
			endpoint directly still returns 403
MAN-03	FR-06	Register a new patient	`PAT-YYYY-NNNNNN` issued and shown
MAN-04	FR-07	Search for that patient by name	Found; details displayed
MAN-05	FR-13, FR-14	Search an appointment by number	Patient, dentist, treatments, times and status all shown
MAN-06	FR-23	Open each of the five reports	Each returns rows and is readable
MAN-07	FR-19	Generate a bill, then print (Ctrl+P)	Print preview shows only the receipt — navigation and buttons are stripped
MAN-08	FR-21	Change a treatment price as Administrator	New price applies to the next booking; existing appointments keep their snapshot price
MAN-09	FR-22	Add a dentist as Administrator	Appears in the booking screen's dentist list
MAN-10	FR-25	Open Help	Explains each function in plain language
MAN-11	FR-26	Sign out, then press Back	Returns to the sign-in screen; the session cannot be resumed
MAN-12	NFR-08	Search with the seeded data	Result returns in well under 2 seconds
MAN-13	NFR-09, FR-11	Attempt a clashing booking	The refusal names the conflict **and** offers three alternative times
MAN-14	—	Toggle light / dark / system appearance	All screens remain legible; the choice survives a page reload

A.8. Defects found and fixed during testing

Recording the defects testing actually caught is stronger evidence that the tests do something than any coverage figure.

#	Found by	Defect	Fix
1	`05_views.sql` on MySQL 8	Three views failed `ONLY_FULL_GROUP_BY`, which MySQL 8 enables by default and MariaDB does not	Every non-aggregated column added to `GROUP BY`; all scripts re-run under MySQL 8's exact default `sql_mode`

#	Found by	Defect	Fix
2	`01_schema.sql` on MySQL 8	`last_value` became a reserved word in MySQL 8 (window functions)	Renamed to `last_issued`; all 72 identifiers checked against the MySQL 8 reserved-word list
3	Browser rendering	The sign-in screen rendered on top of the whole application: `.login-shell { display: grid }` overrode the `hidden` attribute	`[hidden] { display: none !important; }` — caught only by opening the page, never by reading the code
4	Browser rendering, dark mode	Toast messages were unreadable: `background: var(--ink)` inverts to near-white behind white text	Dedicated `--toast-bg` / `--toast-fg` tokens defined per theme
5	`AppointmentServiceTest`	The buffer boundary case was first written at 09:35, which is not on the 15-minute grain, so it failed validation before ever reaching the clash rule — the test was not testing what it claimed	Boundary moved to the adjacent legal slots, 09:30 (refused) and 09:15 (accepted)
6	`AuthenticationServiceTest` on Windows	**A flaky test.** `tokenExpires` set the session window to zero minutes, stamping the expiry at exactly "now". Whether the subsequent check saw that as past depended on the platform clock's resolution: it passed consistently on Linux, but on Windows, where `LocalDateTime.now()` can return the same value on two consecutive calls, it failed intermittently. The suite was green on one operating system and red on another for the same commit	The window is now set to a value that has *already* elapsed (`-1`), so the token is stamped a minute in the past and the race cannot occur. A `Thread.sleep` was rejected: it would have hidden the race behind a delay rather than removing it, and slow tests are the first thing a team stops running

Appendix B — Full design class diagram

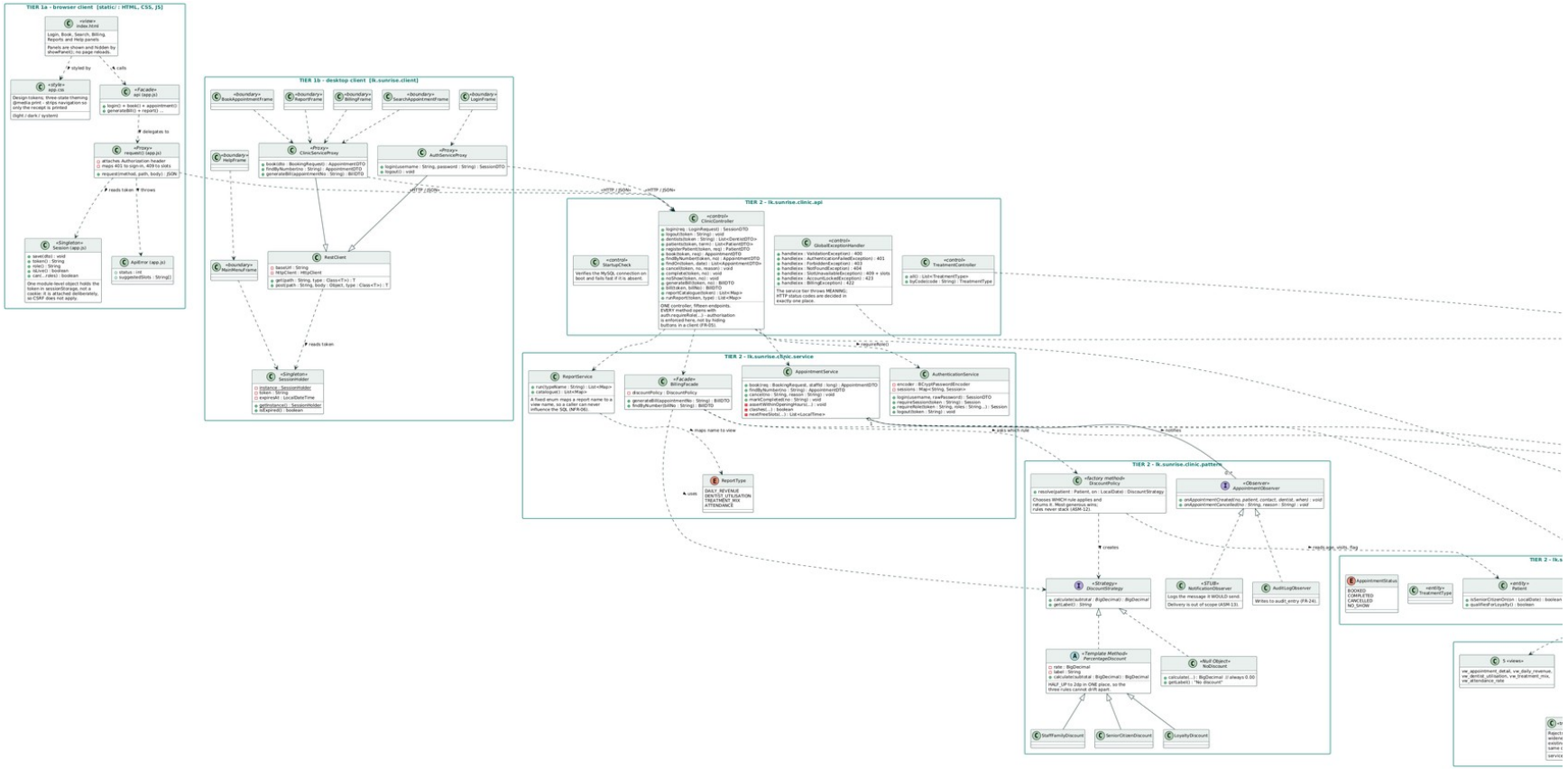


Figure 35: The complete three-tier design class diagram, generated from the implemented system. Two clients, one API, and the business rules that live in MySQL.